

Week5.5 云虚拟机，Shell&本地大语言模型

Updated 2026-03-11 14:18 GMT+8

Compiled by Hongfei Yan (2025 Fall)

<https://github.com/GMyhf/2026spring-cs201/>

logs:

如果用Chrome浏览器访问md文件，可以安装 [Markdown Viewer](#) 插件。安装好之后，点击浏览器右上角 m 图标，Advanced option, allow access打开；点 Settings，选上mathjax, mermaid, syntax, toc。

2025/9/11 删除了春季的云虚拟机，重新创建，增加 1.4节 “在云虚拟机部署《从零构建大模型》代码”

快速上手大语言模型，让AI成为你的学习助手。

拥抱大模型技术，深入本地化实践。在 Mac Studio（M1 Ultra, 64GB）上成功运行70B规模模型，系统已接近性能极限，风扇持续高转，设备温度明显上升。

自主学习平台：登录 小北智学平台（<https://zx.pku.edu.cn>），找到 “数据结构与算法B（闫宏飞）” 课程卡片并加入，即可开启 AI 助教问答式自学。



截止2025年8月：配置为两台 H20 服务器，分别负责 DeepSeek 的 R1 和 V3 模型，单台服务器的并发数均为50

2025/9/3，小北智学底层LLM升级至 **DeepSeek-V3.1**，该版本是2025年8月21日发布。

课程知识库：包含教材、课件、题解等资源，便于自主学习。学习中如有问题，可随时向 AI 助教提问，例如：

- “详细总结知识库内容”
- “课程大纲有哪些？”
- “课程内容是什么？”

0 热身题目

2026spring 数算 (DS Algo) 每日选作				
Updated 2026-03-09 23:45 GMT+8 Compiled by Hongfei Yan (2026 Spring)				
https://github.com/GMyhf/2026spring-cs201/blob/main/DSA_problem_list_at_2026spring.md				
题解, https://fuynaloft.github.io/sol101/ 				
日期	问题编号与名称	标签	难度	链接
0314	20018:蚂蚁王国的越野跑	merge sort, binary indexed tree, binary search	Medium2	http://cs101.openjudge.cn/practice/20018/
			Medium	
0313	30201: 旅行售货商问题	bitmask dp	Tough	http://cs101.openjudge.cn/practice/30201/
			Medium	
0312	1888.使二进制字符串字符交替的最少反转次数	sliding window	Medium2	https://leetcode.cn/problems/minimum-number-of-flips-to-make-the-binary-string-alternating/
			Medium	
0311	2195.Eldiot First Search	dfs and similar, dp, trees	Tough	https://codeforces.com/problemset/problem/2195/E
0311	1364A.XXXXX	two pointers	1200	https://codeforces.com/problemset/problem/1364/A
0310	647.回文子串	two pointers/中心扩散, dp/马拉车	Medium	https://leetcode.cn/problems/palindromic-substrings/
0310	M304.二维区域和检索 - 矩阵不可变	prefix sum	Medium	https://leetcode.cn/problems/range-sum-query-2d-immutable/
0309	1545. 找出第 N 个二进制字符串中的第 K 位	dfs	Medium	https://leetcode.cn/problems/find-kth-bit-in-nth-binary-string/
0309	E303.区域和检索 - 数组不可变	prefix sum	Easy	https://leetcode.cn/problems/range-sum-query-immutable/
0308	01019: Number Sequence	binary search	Tough	http://cs101.openjudge.cn/practice/01019/
0308	20169:排队	DSU	Easy	http://cs101.openjudge.cn/practice/20169/
0307	07207:神奇的幻方	matrix	Medium	http://cs101.openjudge.cn/practice/07207/
0307	05467:多项式加法	dict	Easy	http://cs101.openjudge.cn/practice/05467/
0306	1461.检查一个字符串是否包含所有长度为 K 的二进制子串	bit manipulation	Medium	https://leetcode.cn/problems/check-if-a-string-contains-all-binary-codes-of-size-k/
0306	3827.统计单比特整数	bit manipulation	Easy	https://leetcode.cn/problems/count-monobit-integers/
0305	1404.将二进制表示减到 1 的步骤数	bit manipulation	Medium	https://leetcode.cn/problems/number-of-steps-to-reduce-a-number-in-binary-representation-to-one/
0305	868.二进制间距	bit manipulation	Easy	https://leetcode.cn/problems/binary-gap/
0304	1680.连接连续二进制数字	bit manipulation	Medium	https://leetcode.cn/problems/concatenation-of-consecutive-binary-numbers/
0304	1356.根据数字二进制下 1 的数目排序	bit manipulation	Easy	https://leetcode.cn/problems/sort-integers-by-the-number-of-1-bits/
0303	155.最小栈	OOP, 辅助栈	Medium	https://leetcode.cn/problems/min-stack/
0303	190.颠倒二进制位	bit manipulation	Easy	https://leetcode.cn/problems/reverse-bits/
0302	27300:模型整理	sortings, AI	Mediiium	http://cs101.openjudge.cn/pctbook/M27300
0302	27653: Fraction类	OOP	Easy	http://cs101.openjudge.cn/pctbook/E27653/

27300: 模型整理

sortings, AI, <http://cs101.openjudge.cn/practice/27300/>

深度学习模型（尤其是大模型）是近两年计算机学术和业界热门的研究方向。每个模型可以用“模型名称-参数量”命名，其中参数量的单位会使用两种：M，即百万；B，即十亿。同一个模型通常有多个不同参数的版本。

例如，Bert-110M，Bert-340M 分别代表参数量为 1.1 亿和 3.4 亿的 Bert 模型，GPT3-350M，GPT3-1.3B 和 GPT3-175B 分别代表参数量为 3.5 亿，13 亿和 1750 亿的 GPT3 模型。

参数量的数字部分取值在 [1, 1000) 区间（一个 8 亿参数的模型表示为 800M 而非 0.8B，10 亿参数的模型表示为 1B 而非 1000M）。

计算机专业的学生小 A 从网上收集了一份模型的列表，他需要将它们按照名称归类排序，并且同一个模型的参数量从小到大排序，生成“模型名称: 参数量1, 参数量2, ...”的列表。请你帮他写一个程序实现。

输入

第一行为一个正整数 n ($n \leq 1000$)，表示有 n 个待整理的模型。

接下来 n 行，每行一个“模型名称-参数量”的字符串。模型名称是字母和数字的混合。

输出

每行一个“模型名称: 参数量1, 参数量2, ...”的字符串，符号均为英文符号，模型名称按字典序排列，参数量按从小到大排序。

样例输入

```
1 5
2 GPT-1.3B
3 Bert-340M
4 GPT-350M
5 Bert-110M
6 GPT-175B
```

样例输出

```
1 Bert: 110M, 340M
2 GPT: 350M, 1.3B, 175B
```

提示

tags: string, sort

来源：2023fall zyn

2025/2/14 13:52，大模型可以AC。

<https://chatgpt.com/> 默认是"Search"，选到"Reason"，可以AC。

```
1 def main():
```

```

2     n = int(input().strip())
3     models = {}
4
5     for _ in range(n):
6         s = input().strip()
7         # 按 '-' 分割得到模型名称和参数字符串 (如 "1.3B" 或 "110M")
8         model_name, param_str = s.split('-', 1)
9         # 数字部分为除去最后一个字符 (单位) 得到的部分
10        num_str = param_str[:-1]
11        unit = param_str[-1]
12        # 计算实际数值, B 表示十亿, M 表示百万
13        value = float(num_str) * (1e9 if unit == 'B' else 1e6)
14        # 将模型名称和对应的 (数值, 原始参数字符串) 添加到字典中
15        models.setdefault(model_name, []).append((value, param_str))
16
17        # 按模型名称字典序输出
18        for model_name in sorted(models.keys()):
19            # 同一模型下按照数值从小到大排序
20            sorted_params = sorted(models[model_name], key=lambda x: x[0])
21            # 只取原始参数字符串部分
22            params_str = ", ".join(param for _, param in sorted_params)
23            print(f"{model_name}: {params_str}")
24
25    if __name__ == "__main__":
26        main()
27

```

DeepSeek默认是DeepThink(R1), 能AC。

按照以下步骤来编写Python程序：

1. **读取输入：**首先读取输入模型数量 `n`，然后读取接下来的 `n` 行，每行包含一个模型名称和参数字符串。
2. **解析和存储：**将每个模型名称和参数量解析出来，并存储在一个字典中，字典的键是模型名称，值是一个列表，存储该模型的所有参数量。
3. **排序：**对字典中的模型名称按字典序排序，对每个模型的参数量列表按参数量从小到大排序。
4. **输出：**按照要求的格式输出每个模型及其参数量。

```

1     def parse_model(s):
2         # 将字符串解析为模型名称和参数量
3         parts = s.split('-')
4         model_name = parts[0]
5         param = parts[1]
6         return model_name, param
7
8     def sort_params(params):
9         # 对参数量进行排序
10        # 将参数量转换为统一的单位 (M或B) 进行比较
11        def get_value(param):
12            if param[-1] == 'M':

```

```

13         return float(param[:-1])
14     elif param[-1] == 'B':
15         return float(param[:-1]) * 1000
16     else:
17         return 0 # 无效参数
18
19     return sorted(params, key=get_value)
20
21 def main():
22     n = int(input())
23     models = {}
24
25     # 读取并解析输入
26     for _ in range(n):
27         s = input().strip()
28         model_name, param = parse_model(s)
29         if model_name not in models:
30             models[model_name] = []
31         models[model_name].append(param)
32
33     # 对模型名称按字典序排序
34     sorted_models = sorted(models.keys())
35
36     # 对每个模型的数量进行排序
37     for model in sorted_models:
38         sorted_params = sort_params(models[model])
39         # 输出结果
40         print(f"{model}: {' '.join(sorted_params)}")
41
42 if __name__ == "__main__":
43     main()

```

1 虚拟机

虚拟机 (Virtual Machine, VM) 是一种通过软件模拟的、具有完整硬件系统功能的计算机系统，运行在一个完全隔离的环境中。虚拟机可以运行在物理计算机之上，允许用户在同一台硬件上运行多个操作系统和应用程序，极大地提高了资源利用率和灵活性。

常见的本地虚拟机软件

1. **Parallels Desktop**: 主要为Mac用户提供了一个运行其他操作系统（如Windows、Linux等）的解决方案。
2. **VirtualBox**: 是一款开源的虚拟化产品，由Oracle公司提供支持。它可以安装在多种操作系统上（如Windows、Linux等），并能够运行大量的客户操作系统。VirtualBox因其免费受到广泛欢迎。

云端虚拟机

- **clab.pku.edu.cn**: CLab 是服务北大师生的云计算平台。提供基于云的虚拟实验室环境，供学生 and 研究人员用于教学、学习和科研目的。用户可以通过互联网访问这些虚拟机，执行编程实验、模拟等任务。

写程序时，通常打开终端（terminal），通过 SSH 登录到服务器，使用 vi 编辑代码，并借助 python/g++/gcc 进行解释、编译和调试。

现在，北京大学为大家提供了这样的实际环境——每位同学和老师都可以领取并使用一台云端虚拟机。该虚拟机配置为 4GB 内存、100GB SSD 硬盘，支持安装 Linux 系统，供大家自由探索和学习。

👉 申请地址：<https://clab.pku.edu.cn/>

💬 答疑渠道：QQ群 432191140，入群问题答案：linuxclub@pku.edu.cn

无论是本地还是云端的虚拟机，它们都提供了灵活的计算资源分配方案，帮助用户测试软件、开发新应用或进行研究工作，而无需投资额外的硬件设施。随着云计算技术的发展，越来越多的服务迁移到了云端，使得用户可以从任何地方访问高性能的计算资源。

Q1. 部署虚拟机意义？

既然第三步大模型安装和测试，可以不用虚拟机，这一步部署虚拟机意义？

部署虚拟机提供了一个分布式计算环境，在这个环境中每个虚拟机都可以作为一个独立的计算节点运行。当你有一个需要大量计算资源的任务时（比如分治任务），可以将这个任务分解成多个较小的子任务，并将这些子任务分配给不同的虚拟机来并行处理。这样做的好处是可以大幅减少总计算时间，因为多个子任务可以同时在不同的机器上执行。

利用部署的虚拟机集群来执行具体的计算任务。具体来说，一旦所有虚拟机都设置好并且可以通过 SSH 访问（即公钥已经添加到各个虚拟机的 `authorized_keys` 文件中），就可以通过编写脚本自动登录各个虚拟机、分发任务以及收集结果。

100个选课学生创建的虚拟机可以形成一个分布式系统

Clab.pku.edu.cn 云虚拟机，为每个用户提供 4 CPU, 4 GB RAM, 100 GB Disk。每个虚拟机的 `.ssh/authorized_keys`，保存了可以 ssh 登录虚拟机的公钥。我们班 100 人，共 440 CPU, 440 GB, 11000 GB Disk。如果大家互相把每人的公钥（保存在各位本地机器 `.ssh/id_ed25519.pub` 中的字符串），加入虚拟机的 `authorized_keys`，则 100 个虚拟机可以形成一个分布式系统，可以用来计算分治任务。

分治任务是一种将问题分解成更小的子问题，分别求解这些子问题，然后合并这些子问题的解来得到原问题解的方法。一个典型的例子是计算一个大数组中所有元素的和。我们可以将这个数组分割成若干个较小的数组，每个虚拟机负责计算其对应的小数组的和，最后再将这些结果汇总起来得到整个数组的和。

登录云端服务器并利用云端计算资源，是现代开发和计算任务中常见的工作方式。一旦掌握了相关技能，便可以高效地使用云端服务器，拓展更多应用场景和计算任务。相比之下，本地设备通常性能有限，更适用于日常开发和基础调试。

部署虚拟机的意义主要体现在以下几个方面：

1. 与云端环境接轨，培养云计算使用习惯

- 现代 AI 计算通常依赖云端 GPU 资源（如 AWS、Google Cloud、Azure），本地机器性能有限，无法高效运行大模型。
 - 通过虚拟机模拟远程服务器环境，让大家**熟悉 SSH 登录、环境配置、远程代码执行等操作**，为后续使用云端资源打下基础。

2. 隔离环境，避免污染本地系统

- 大模型部署涉及大量 Python 依赖（如 CUDA、PyTorch、Transformers），可能与本地已有环境冲突。
- 在虚拟机或 Docker 容器中运行，可以隔离依赖，避免影响日常工作环境。

3. 统一环境，减少兼容性问题

- 本地机器硬件和系统差异较大（Windows/Linux/Mac），直接安装可能遇到驱动、CUDA 版本兼容性问题。
- 通过虚拟机，大家可以在统一的 Linux 服务器环境下测试，确保配置一致，提高稳定性。

4. 便于迁移到云端服务器

- 如果在本地虚拟机上调试成功，可以无缝迁移到真正的云服务器，而无需重新配置环境。
- 这样可以降低云端服务器的调试成本，提高使用效率。

结论 即使本地能跑通大模型，使用虚拟机仍然有 **环境隔离、与云端兼容、避免污染本机、提高可移植性** 等作用。部署虚拟机不仅是为了当前测试，更是为未来高效使用云端计算资源做准备。

Q2.时间复杂度和空间复杂度

Q2. 推荐力扣每日一题，简单、中等的都挺好。力扣的简单题目，力争提交后，击败超过 50%。

感觉力扣波动挺大的，有的时候差一两毫秒就是10%和80%的区别，两次提交一样的代码就能差三四毫秒。

A. 这是**事后测量**，其结果受到多个因素的影响，包括输入数据规模、编程语言的执行效率、以及系统当时的负载情况（如是否有其他任务在运行）。由于这些因素具有不确定性，测量结果难以精准预测，因此更常采用**事前估计**的方法，即**大 O 表示法 (Big-O Notation)**。它主要用于分析算法的时间复杂度和空间复杂度。

在算法优化中，**时间复杂度**和**空间复杂度**通常难以同时达到最优，优化策略往往遵循“**以空间换时间**”的原则，而不是“**以时间换空间**”。这是因为：

1. **空间资源相对廉价且可回收**：随着硬件的发展，存储成本不断降低，而计算时间却依然是关键瓶颈。
2. **内存可复用**：算法执行完毕后，占用的内存可以释放，再用于后续任务，因此合理使用额外空间（如哈希表、缓存）通常是值得的。

因此，在优化算法时，应优先考虑通过增加适量的空间占用（如使用缓存、预计算等），来减少计算时间，从而提升整体运行效率。

LeetCode 的提交排名确实有较大的波动，这是由多个因素导致的，包括：

1. **服务器负载**：LeetCode 的评测服务器可能在不同时间段运行不同的任务，影响执行时间。
2. **输入数据**：即使是同一个测试用例，底层执行可能受到缓存、内存分配等因素的影响。
3. **编程语言**：C++、Java、Python 的执行效率不同，比如 Python 由于解释执行，通常比 C++ 慢。

4. **JIT 优化**：某些语言（如 Java、PyPy）可能在运行过程中进行 Just-In-Time (JIT) 编译，导致运行时间有所浮动。
5. **CPU 调度**：服务器运行多个代码提交，CPU 资源可能被分配给其他任务，影响你的代码执行时间。

如何更稳定地优化代码？

因为测不准原理（执行时间有波动），我们不能只依赖测量结果，而是要使用事前估计的方法，即 **时间复杂度分析** (Big-O notation)。

- **时间复杂度** (Time Complexity)：分析算法的执行时间随输入规模 n 增长的变化，例如：
 - 线性时间 $O(n)$ ：遍历数组。
 - 对数时间 $O(\log n)$ ：二分查找。
 - 二次时间 $O(n^2)$ ：双层循环。
 - 指数时间 $O(2^n)$ ：递归爆炸增长（如暴力搜索）。
- **空间复杂度** (Space Complexity)：分析算法所需的额外内存。例如：
 - $O(1)$ ：仅使用几个变量。
 - $O(n)$ ：存储数组或哈希表。
 - $O(n^2)$ ：存储邻接矩阵。

LeetCode 提交如何击败 50%+?

1. **优化时间复杂度**：优先选用更优的算法。例如：
 - 哈希表代替嵌套循环（从 $O(n^2)$ 优化为 $O(n)$ ）。
 - 二分查找代替遍历（从 $O(n)$ 优化为 $O(\log n)$ ）。
 - 动态规划优化递归（避免指数增长）。
2. **减少不必要的计算**：
 - 缓存计算结果（如 Memoization）。
 - 提前终止循环（如 `break`、`continue`）。
 - 避免重复计算（如 `set` 记录已访问值）。
3. **选择合适的数据结构**：
 - 查找问题：用哈希表 (`dict` / `unordered_map`) 代替数组遍历。
 - 队列 / 栈：BFS / DFS。
 - 堆：求前 K 大 / K 小值。
4. **语言优化技巧**：
 - **Python**：使用 `map()`、`zip()`、`sum()` 等内置函数，避免手写循环。
 - **C++**：`vector` 预分配 (`reserve()`)，避免动态扩容。
 - **Java**：`StringBuilder` 代替 `String + String`，减少字符串拼接开销。

总结：如果代码能在理论上达到最优复杂度（如从 $O(n^2)$ 降到 $O(n)$ ），即使排名偶尔波动，长期来看仍能稳定击败 50% 以上的提交。

1.1 创建云端虚拟机

访问 <https://clab.pku.edu.cn>

⚠️：要用好云端虚拟机，需要熟悉Linux Shell命令，可以参考之后节内容 [2 Linux Shell 使用](#)

同时打开入门文档，<https://clab.pku.edu.cn/docs/getting-started/introduction>

我是在mac机器操作

在terminal中查看是否有公钥，

```
1 | ls .ssh/id_ed25519.pub
```

如果没有，运行

```
1 | ssh-keygen -t ed25519
```

- 保密通信：发送方 用 接收方的公钥 加密 -> 接收方 用 自己的私钥 解密。
- 身份验证（数字签名）：发送方 用 自己的私钥 签名 -> 接收方 用 发送方的公钥 验证签名。

点击“云主机”，点击“创建云主机”按钮

可用域：nova

架构：X86

类别：labs_and_courses, l3, 4CPU, 4.00GiB

启动源：镜像

操作系统：RockyLinux 9.5

Clab PKU 推荐的镜像是 RockyLinux 9，是一个基于 RHEL 的 Linux 发行版，有着良好的兼容性和稳定性。Ubuntu 24.04.1 和 Ubuntu 20.04 也是非常好的选择，有着良好的社区支持。

对于新手来说，Ubuntu 或 Linux Mint 可能是最好的起点，而对于寻求最新技术和功能的用户，Fedora 或 Arch Linux 则可能是更好的选择。对于企业级应用，CentOS Stream 或 openSUSE Leap 可以提供所需的支持和稳定性。

从云硬盘启动：是

系统盘：类型SSD，容量100GiB。直接用满100GB，这样省事，否则之后还要挂载数据盘。



点击页面右下角的“下一步”按钮，进入网络设置

共享网络：pku-new

虚拟网卡：默认值

安全组：默认值

点击页面右下角的“下一步”按钮，进入名称和密钥设置

名称：YouNameOne

登录凭证：默认值

SSH密钥对：点“导入密钥”，名称：YouNameOne，公钥：把 `.ssh/id_ed25519.pub` 文件中内容贴进来

确认云主机的配置

Q3.unix上ssh登录，与RSA的公钥密钥关系？

在Unix系统上使用SSH进行登录时，RSA公钥和私钥的关系是非常重要的，因为它们是实现安全远程登录的基础。下面我将详细解释这个关系，并引用相关资料来支持说明。

SSH登录与RSA密钥对

当用户希望通过SSH协议安全地连接到远程服务器时，可以采用两种主要的认证方式：口令认证和基于密钥的认证（也称为公钥认证）。这里我们关注的是基于密钥的认证，因为它更安全且便捷。

公钥认证流程

1. **生成密钥对**：首先，在客户端机器上，用户需要生成一对RSA密钥（或其它非对称加密算法的密钥对），这可以通过 `ssh-keygen -t rsa` 命令完成。该命令会生成一个私钥文件（默认为 `~/.ssh/id_rsa`）和一个对应的公钥文件（默认为 `~/.ssh/id_rsa.pub`）。
2. **部署公钥**：然后，用户的公钥需要被添加到想要登录的远程服务器上的 `~/.ssh/authorized_keys` 文件中。这可以通过手动复制粘贴或者使用 `ssh-copy-id user@remote_host` 命令自动化完成。
3. **发起连接请求**：当用户尝试通过SSH连接到服务器时，客户端会发送其公钥给服务器进行验证。
4. **服务器验证**：服务器收到公钥后，会在 `authorized_keys` 文件中查找是否有匹配的公钥。如果有匹配项，则服务器会生成一个随机字符串并用找到的公钥加密后发送给客户端。
5. **客户端解密**：客户端收到加密后的字符串后，使用自己的私钥对其进行解密，并将解密后的结果发回给服务器。
6. **确认身份**：如果服务器能够验证返回的结果确实是之前发送的随机字符串，则认为客户端的身份得到了验证，允许登录。

RSA公钥和私钥的作用

- **公钥**：用于加密数据，任何人都可以获得它，因为它不包含任何敏感信息。在SSH场景下，公钥被用来加密服务器生成的挑战字符串。

- **私钥**：必须保密，只有持有者知道。私钥用于解密由公钥加密的数据。在SSH场景下，私钥被用来解密来自服务器的加密挑战字符串。

安全性考虑

由于RSA加密算法的安全性依赖于大整数分解的困难性，因此确保私钥的安全至关重要。即使公钥公开，没有对应的私钥也无法解密数据。此外，为了进一步提高安全性，通常建议使用较长的密钥长度（如2048位或更高），以增加破解难度。

综上所述，SSH登录过程中使用的RSA公钥和私钥遵循非对称加密的基本原则，即公钥用于加密，私钥用于解密，这一机制保障了通信双方的身份验证和数据传输的安全性。

1.2 连接云主机

云主机创建完成后，可以点击云主机的名称进入云主机详情页面。在这里可以看到云主机的状态、IP 地址等信息。我的IP是 `10.129.242.57`。

The screenshot displays the cloud console interface for a specific cloud instance. On the left is a dark sidebar with navigation options like '首页', '计算', '云主机', '云主机快照', etc. The main area shows the '云主机详情' (Cloud Instance Details) page for an instance named 'jensen'. At the top, the ID is '3c028212-b490-4c38-ab29-db314ad53f79'. Below this, a table lists basic info: name 'jensen', status '运行中' (Running), creation time '2025-09-10 23:04:29', and host '-'. A secondary table lists tabs: '详情' (selected), '云硬盘', '云主机快照', '网卡', '浮动IP', '安全组', '操作日志', and '日志'. The '详情' tab is active, showing four sections: '网络信息' (Network Info) with IP '10.129.242.57', '配置信息' (Configuration Info) with 4 GiB memory and 4 vCPUs, '镜像信息' (Image Info) for RockyLinux 9.5, and '安全组信息' (Security Group Info) set to 'default'. On the right, the '云主机架构图' (Cloud Instance Architecture Diagram) shows the instance connected to a network and a disk. The disk details are: ID '8e9980de-b701-45a7-bb22-cd7b63a8c7ec', capacity '100GiB', type 'SSD', and creation time '14小时前'.

在terminal中登录云主机

```
1 | ssh rocky@10.129.242.57
```

输入yes，回车

在云端虚拟机中登陆网关，访问外网



Q4. vi的使用?

需要会用vi编辑器，编辑文件。 <https://www.runoob.com/linux/linux-vim.html>

vim

Vim (Vi IMproved), a command-line text editor, provides several modes for different kinds of text manipulation.

Pressing i in normal mode enters insert mode. Pressing `g` goes back to normal mode, which enables the use of Vim commands.

See also: vimdiff, vimtutor, nvim.

More information: <https://www.vim.org>.

- Open a file:

`vim path/to/file`

- Open a file at a specified line number:

`vim +line_number path/to/file`

- View Vim's help manual:

`:help`

- Save and quit the current buffer:

`ZZ|:x|:wq`

- Enter normal mode and undo the last operation:

`u`

```
/search_pattern
```

```
:%s/regular_expression/replacement/g
```

```
:set nu
```

在 Linux 上，你可以安装完整版 Vim:

```
1 sudo apt install vim          # Ubuntu/Debian
2 sudo dnf install vim          # Fedora
3 sudo yum install vim          # CentOS
4 brew install vim              # macOS (使用 Homebrew)
```

```
hfyam — rocky@jensen:~ — ssh rocky@10.129.242.98 — 165x41
```

```
[help.txt] For Vim version 8.2. Last change: 2020 Aug 15

VIM - main help file

Move around: Use the cursor keys, or "h" to go left,          k
              "j" to go down, "k" to go up, "l" to go right.    h l
Close this window: Use ":q<Enter>".                               j
Get out of Vim:   Use ":qa!<Enter>" (careful, all changes are lost!).

Jump to a subject: Position the cursor on a tag (e.g. bars) and hit CTRL-].
With the mouse:   ":set mouse=a" to enable the mouse (in xterm or GUI).
                  Double-click the left mouse button on a tag, e.g. bars.
Jump back:        Type CTRL-O. Repeat to go further back.

Get specific help: It is possible to go directly to whatever you want help
on, by giving an argument to the :help command.
Prepend something to specify the context: help-context

WHAT      PREPEND  EXAMPLE
Normal mode command      :help x

[No Name]
"help.txt" [readonly] 239L, 8899B
```

```
1 #!/usr/bin/env python3
```

```

2
3 import requests
4 import getpass
5
6 # 从命令行获取用户名和密码
7 username = input("请输入用户名: ")
8 password = getpass.getpass("请输入密码: ")
9
10 url = "https://its4.pku.edu.cn/cas/ITSCClient"
11 payload = {
12     'username': username,
13     'password': password,
14     'iprange': 'free',
15     'cmd': 'open'
16 }
17 headers = {'Content-type': 'application/x-www-form-urlencoded'}
18
19 result = requests.post(url, params=payload, headers=headers)
20 print(result.text)

```

将程序保存为 `login.py`，运行程序，根据提示输入用户名和密码，就可以登陆网关了。

运行程序

```
1 | python login.py
```

要使用Vi编辑器粘贴并保存这段Python程序，并最终执行它，请按照以下步骤操作：

步骤 1: 打开终端

首先，打开你的Linux终端。

步骤 2: 使用Vi创建新文件

在终端中输入以下命令来使用Vi创建一个名为 `login.py` 的新文件：

```
1 | vi login.py
```

步骤 3: 进入插入模式

进入Vi编辑器后，按下键盘上的 `i` 键进入插入模式（Insert Mode）。此时，你可以开始编辑文件内容了。

步骤 4: 粘贴代码

将提供的Python代码复制到剪贴板中，然后在Vi编辑器内右键点击选择“粘贴”或者直接使用快捷键 `Ctrl+Shift+V` 来粘贴代码。确保所有代码都被正确地粘贴到了文件中。

步骤 5: 保存文件

完成代码粘贴后，按下 `Esc` 键退出插入模式。然后输入以下命令保存文件并退出Vi编辑器：

```
1 | :wq
```

这里的 `:` 表示进入命令模式, `w` 是写入 (保存) 文件, `q` 是退出Vi编辑器。

步骤 6: 赋予执行权限

为了能够运行这个Python脚本, 你可能需要给它赋予执行权限。在终端中输入以下命令:

```
1 | chmod +x login.py
```

步骤 7: 执行程序

最后, 在终端中输入以下命令来运行这个Python程序:

```
1 | python3 login.py
```

注意: 根据你的系统配置和安装的Python版本, 可能需要使用 `python3` 而不是 `python` 来运行脚本。

现在, 根据提示输入用户名和密码, 就可以尝试登录网关了。

Q5.在输入密码的时候其它键都没反应?

在输入密码的时候其它键都没反应, 只能敲回车然后跳出密码错误该怎么办?

```
[rocky@xwk ~]$ python3 login2.py
请输入用户名: 2400011466
请输入密码:
{"error": "密码错误, 请重新输入。"}

```

A. 输入密码的时候不显示而已, 正常输入就可以, 这是为了保护你的密码

1.3 增加交换分区, 扩展系统的可用内存

如果4GB内存够用, 可以跳过此步。

```
1 | sudo dd if=/dev/zero of=/swapfile bs=1M count=6144
2 | sudo chmod 600 /swapfile
3 | sudo mkswap /swapfile
4 | sudo swapon /swapfile
5 | sudo vi /etc/fstab
6 |   在最后加一行 /swapfile none swap sw 0 0
7 | sudo mount -a
8 |

```

通过上述操作所创建和配置的6GB是硬盘模拟的内存。

虚拟内存是计算机系统内存管理的一种技术。它使得应用程序认为它拥有连续的可用的内存 (一个连续完整的地址空间), 而实际上, 它通常是被分隔成多个物理内存碎片, 还有部分暂时存储在外部的磁盘存储器上, 在需要时进行数据交换。

具体解释本次操作创建的虚拟内存

- 创建交换文件：`sudo dd if = /dev/zero of=/swapfile bs = 1M count = 6144` 这条命令创建了一个大小为6GB（ $1M \times 6144 = 6GB$ ）的文件 /swapfile，这个文件的内容全部是0（因为 `if=/dev/zero`）。这个文件就是用来模拟内存空间的基础。
- 设置权限：`sudo chmod 600 /swapfile` 设置文件权限，确保只有所有者可以读写该文件，保障数据安全。
- 格式化为交换空间：`sudo mkswap /swapfile` 将创建的文件格式化为交换空间格式，使其能够被系统识别和使用。
- 启用交换文件：`sudo swapon /swapfile` 激活这个交换文件，让它开始充当虚拟内存的角色。此时，系统在物理内存不足时，就会将部分暂时不使用的数据存储到这个交换文件中。
- 配置开机自动挂载：通过编辑 /etc/fstab 文件并执行 `sudo mount -a`，确保系统重启后，这个交换文件依然能够自动挂载并生效。

当系统的物理内存不够用时，操作系统会将一部分暂时不使用的数据从物理内存移动到虚拟内存（即 /swapfile 文件）中，从而为当前需要运行的程序腾出物理内存空间。当这些数据再次需要被使用时，再从虚拟内存中调回到物理内存。

不过，虚拟内存的读写速度比物理内存慢很多，因为涉及到磁盘I/O操作。所以，在实际应用中，合理配置物理内存和虚拟内存的比例，对于系统的性能优化非常重要。

虚拟内存

虚拟内存是一种内存管理技术，它允许操作系统为每个进程提供一个独立、连续的地址空间，而无需考虑实际物理内存的布局或限制。通过虚拟内存，系统可以将数据存储在物理内存（RAM）和硬盘上的交换分区之间动态移动，以优化内存使用效率。

交换分区

交换分区（或交换文件）是虚拟内存体系中的一个组成部分，主要用于扩展系统的可用内存。当系统的物理内存（RAM）不足时，不常用的内存页会被暂时移到交换分区上，从而释放物理内存供其他进程使用。这个过程称为“交换”（swapping）。因此，交换分区实际上是作为物理内存的一种补充，用于临时存放那些当前不活跃的数据。

关系

- **交换分区是实现虚拟内存的一部分**：虽然交换分区本身不是虚拟内存，但它对于支持虚拟内存机制至关重要。它是虚拟内存系统用来在物理内存不足时存储数据的地方。
- **虚拟内存不仅仅包含交换分区**：虚拟内存还包括物理内存。虚拟内存系统会根据需要在物理内存和交换分区之间移动数据，以维持系统的高效运行。
- **透明性**：对用户和应用程序而言，虚拟内存提供了统一的地址空间抽象，使得是否使用交换分区对它们来说是透明的。无论是位于物理内存还是交换分区上的数据，都通过相同的虚拟地址进行访问。

在Linux系统中，虚拟内存并不对应硬盘上连续的单元。虚拟内存是一种内存管理技术，允许操作系统以为每个进程都有一个独立、连续且私有的地址空间的方式运行程序，但实际上这些地址空间可能映射到物理内存（RAM）的不同部分或硬盘上的交换分区（Swap space）。

虚拟内存的关键点包括：

- **非连续性**：虚拟内存地址与物理内存地址之间没有直接的一对一关系。操作系统通过页表

(Page Table) 来维护虚拟地址到物理地址之间的映射。这意味着即使虚拟地址空间看起来是连续的，它也可以映射到物理内存中的非连续区域。

- **分页机制**：虚拟内存通常以固定大小的块（称为“页”，一般为4KB）进行管理，并与物理内存中的相应块（也称为“帧”）进行映射。当物理内存不足时，某些页面会被保存到硬盘上的交换分区（Swap space），以便释放物理内存供其他更需要的进程使用。
- **交换分区（Swap space）**：虽然交换分区位于硬盘上，但它并不是用来存储连续的虚拟内存块的。相反，它是作为物理内存的一个扩展，用于临时存放那些当前不活跃的内存页。因此，即使是交换分区本身，也不会保证虚拟内存页是以连续的形式存储的。

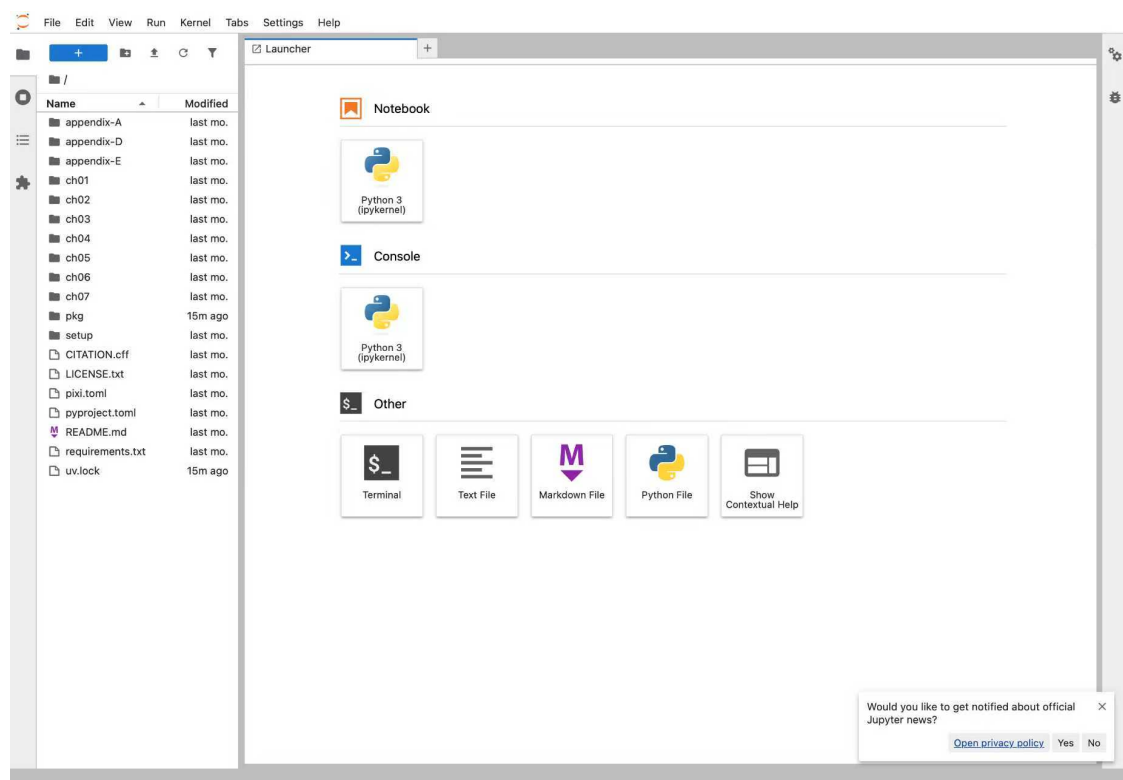
综上所述，虚拟内存的设计和实现确保了即便在物理层面上数据存储是非连续的，也能给用户和应用程序提供一个看似连续、一致的内存视图。这种抽象不仅提高了内存使用的灵活性和效率，还增强了系统的稳定性和安全性。

Q6. 请问成功与虚拟机连接上之后我们下一步要干什么？

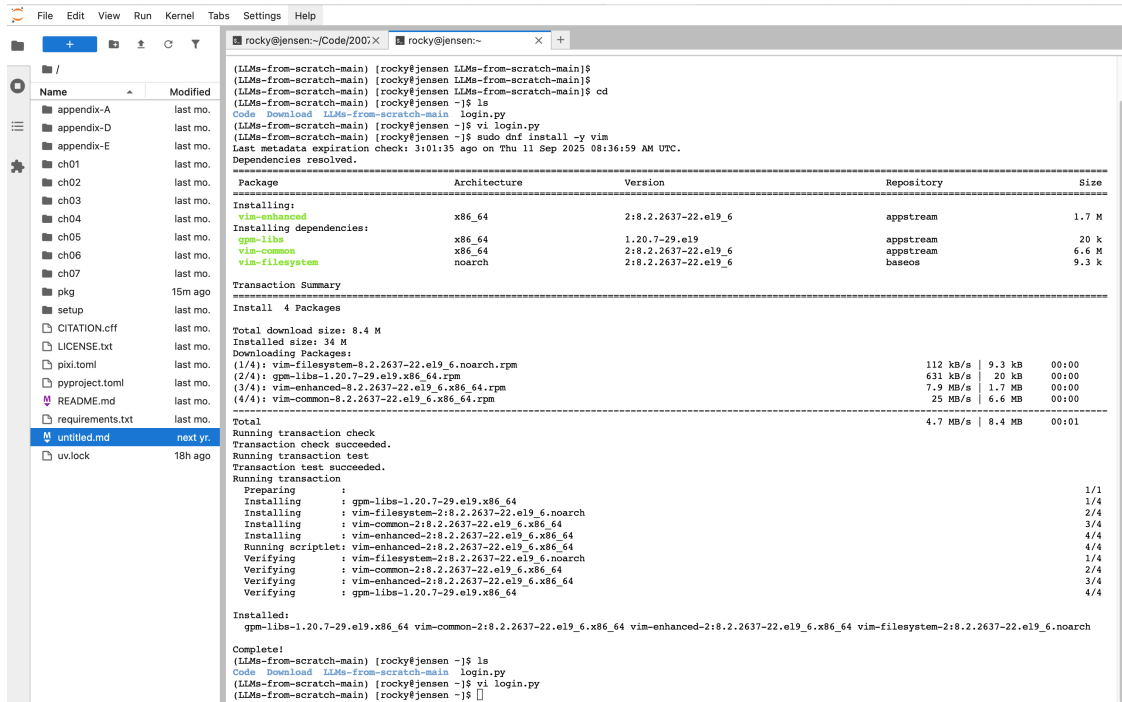
A: 你多了台机器，想到做什么都可以的。这是学校送给你的机器，系统是Linux，可以在上面写程序、编译/解释/调试、运行，或者部署 从零构建模型 的代码等。

例如：

1) 部署 从零构建大模型 的代码。放到clab云虚拟机上了，可以从我本地机器通过浏览器访问。



2) 我现在可以在浏览器里面写程序，实际上就是连到云虚拟机。



这是浏览器的一个页面。类似于力扣提供的 Playground，但是不付费的化，只能开6个页面。我这云虚拟机可以开无数。

3) 把我有的OJ测试数据，也搬到云虚拟机上了。testing_code.py, offlinejudge.zsh都可以用的。

```

1  # testing_code.py
2  import subprocess
3  import difflib
4  import os
5  import sys
6
7  def test_code(script_path, infile, outfile):
8      command = ["python", script_path] # 使用Python解释器运行脚本
9      with open(infile, 'r') as fin, open(outfile, 'r') as fout:
10         expected_output = fout.read().strip()
11         # 启动一个新的子进程来运行指定的命令
12         process = subprocess.Popen(command, stdin=fin,
13                                     stdout=subprocess.PIPE)
14         actual_output, _ = process.communicate()
15         if actual_output.decode().strip() == expected_output:
16             return True
17         else:
18             print(f"Output differs for {infile}:")
19             diff = difflib.unified_diff(
20                 expected_output.splitlines(),
21                 actual_output.decode().splitlines(),
22                 fromfile='Expected', tofile='Actual', lineterm='
'
23             )
24             print('\n'.join(diff))
25             return False
26
27  if __name__ == "__main__":

```

```

28 # 检查命令行参数的数量
29 if len(sys.argv) != 2:
30     print("Usage: python testing_code.py <filename>")
31     sys.exit(1)
32
33 # 获取文件名
34 script_path = sys.argv[1]
35
36 #script_path = "class.py" # 你的Python脚本路径
37 #test_cases = ["d.in"] # 输入文件列表
38 #expected_outputs = ["d.out"] # 预期输出文件列表
39 # 获取当前目录下的所有文件
40 files = os.listdir('.')
41
42 # 筛选出 .in 和 .out 文件
43 test_cases = [f for f in files if f.endswith('.in')]
44 test_cases = sorted(test_cases, key=lambda x: int(x.split('.')[0]))
45 #print(test_cases)
46 expected_outputs = [f for f in files if f.endswith('.out')]
47 expected_outputs = sorted(expected_outputs, key=lambda x:
48 int(x.split('.')[0]))
49 #print(expected_outputs)
50
51 for infile, outfile in zip(test_cases, expected_outputs):
52     if not test_code(script_path, infile, outfile):
53         break

```

testing_code.py只能是测试数据是 0.in, 0.out, 1.in,1.out....这种数字文件名时候才能用，因为代码里面有个按照整数排序。如果测试文件是其他字母名字，不能用。

offlinejudge.zsh

```

1 cd $2
2 for i in *.in; do
3     diff -y <(python3 "$1" < "$i") "${i%.*}.out"
4 done
5

```

```
(base) hfyang@HongfeideMac-Studio data % zsh ../../offlinejudge.e1.py ./
2006.5 2006.5
6211 | 2793
7299.5 | 4216.5
7128 | 5445
6794.5 | 5953
6461 | 5645
5545 | 5362.5
5280 | 4218
5462.5 | 4749
6364 6364
9840 | 8848
9131.5 | 5778.5
8214 | 3134
7815.5 | 6368.5
7417 <
4474 <
3628 <
4474 <
5320 5320
4692 | 3467.5
> 3307
> 3467.5
> 3628
> 3467.5
> 4363
> 4604
4845 4845
5300.5 | 4604
5756 | 4396
5300.5 | 4620.5
4845 4845
5300.5 <
5756 <
```

offlinejudge.zsh都可以用，测试文件什么名字都可以。需要在 测试数据 所在目录中运行。

● ● ●	Book_my_flight_202508.md — 已编辑
大纲	
计算思维算法实践	
> 第1章 计算机科学与智能时代	
> 第2章 Python语言基础与编程环境准备	
> 2.1 Python语言基础	
> 2.2 Python 开发环境配置	
> 2.3 调试辅助工具	
2.3.1 Pythontutor可视化工具	
2.3.2 使用测试数据进行调试	
> 2.4 算法分析	
> 第3章 计算思维实践	
> 3.1 语法类	
> 3.1.1 简单计算	
E02676: 整数的个数	
E02701: 与7无关的数	
E02712: 细菌繁殖	
E02733: 判断闰年	
E02750: 鸡兔同笼	
M19944: 这一天星期几	
> 3.1.2 排序	
E02724: 生日相同	
M01002: 方便记忆的电话号码	
E07618: 病人排队	
> 3.1.3 字符串	
2. 使用测试数据运行程序	
本书提供了一些题目的测试数据，见：	
https://github.com/GMyhf/2021fall-cs101/tree/main/cs101_test_data	
下载并解压后，你会看到类似 <code>0.in</code> , <code>0.out</code> 的文件对。	
• <code>0.in</code> 是输入数据 • <code>0.out</code> 是正确输出结果	
将程序文件与测试数据放在同一目录下，然后在 PowerShell 中运行：	
<pre>1 python 4A.py < 0.in > 0my.out</pre>	
解释：	
• <code>< 0.in</code> 表示将 <code>0.in</code> 内容作为输入数据 • <code>> 0my.out</code> 表示程序的输出结果写入到 <code>0my.out</code> 文件	
此时，你只需对比 <code>0my.out</code> 与 <code>0.out</code> ，即可发现程序与标准答案的差异，从而定位 bug。	
3. 在 macOS 中的操作	
在 macOS 下操作方式类似，也需要先确认 Python 的安装位置，然后使用同样的 <code><</code> 与 <code>></code> 重定向符号来运行和保存结果。	
视频讲解参考：	
• Windows 环境：https://www.bilibili.com/video/BV1jT4y1B7eU • macOS 环境：https://www.bilibili.com/video/BV15341137sg	

本书提供了一些题目的测试数据，见：

https://github.com/GMyhf/2021fall-cs101/tree/main/cs101_test_data

下载并解压后，你会看到类似 `0.in`, `0.out` 的文件对。

- `0.in` 是输入数据
- `0.out` 是正确输出结果

将程序文件与测试数据放在同一目录下，然后在 **PowerShell** 中运行：

```
1 python 4A.py < 0.in > 0my.out
```

解释：

- `< 0.in` 表示将 `0.in` 内容作为输入数据
- `> 0my.out` 表示程序的输出结果写入到 `0my.out` 文件

此时，你只需对比 `0my.out` 与 `0.out`，即可发现程序与标准答案的差异，从而定位 bug。

3. 在 macOS 中的操作

在 **macOS** 下操作方式类似，也需要先确认 Python 的安装位置，然后使用同样的 `<` 与 `>` 重定向符号来运行和保存结果。

视频讲解参考：

- Windows 环境：<https://www.bilibili.com/video/BV1jT4y1B7eU>
- macOS 环境：<https://www.bilibili.com/video/BV15341137sg>

1.4 在云虚拟机部署《从零构建大模型》

《从零构建大模型》代码，<https://github.com/rasbt/LLMs-from-scratch>

RockyLinux / RHEL / Fedora 系统级的包管理器（类似于 Ubuntu 的 `apt`）。可以安装系统软件、库、Python 解释器。

Python 官方的包管理工具。可以安装 Python 生态里的库（numpy、torch、jupyter 等）。

uv 是更高级的 Python 包管理器，创建虚拟环境并安装项目依赖（比 pip 快）。例如：创建虚拟环境：

```
uv venv, 安装依赖: uv pip install -r requirements.txt, 运行程序: uv run python train.py
```

以下安装 Python3.11、uv、依赖、Jupyter。

1. 安装 Python 3.11

RockyLinux 默认可能只有 Python 3.9，需要启用 **EPEL** 和 **CRB**，用 `dnf` 安装：

```
1 | sudo dnf update -y
2 | sudo dnf install -y epel-release
3 | sudo dnf config-manager --set-enabled crb
4 | sudo dnf install -y python3.11 python3.11-devel python3.11-pip
```

确认版本：

```
1 | python3.11 --version
```

2.安装 uv（包管理工具）

和 Mac 一样用 pip：

```
1 | python3.11 -m pip install --upgrade pip
2 | python3.11 -m pip install uv
```

3.获取代码

```
1 | wget https://github.com/rasbt/LLMs-from-scratch/archive/refs/heads/main.zip
2 | unzip main.zip
3 | cd LLMs-from-scratch-main
```

4.创建虚拟环境

```
1 | uv venv --python=python3.11
2 | source .venv/bin/activate
```

验证：

```
1 | which python
2 | python --version
```

应该输出 `.venv/bin/python` 和 `3.11.x`。

5.安装依赖

```
1 | uv pip install torch
2 | uv pip install -r requirements.txt
```

为了之后程序能 import sklearn.xxx,

可以一并安装

```
1 | uv pip install scikit-learn
```

6.设置新口令

```
1 | jupyter server password
```

7.启动 Jupyter Lab

Linux 下和 Mac 一样：

```
1 | uv run jupyter lab --no-browser --ip=0.0.0.0 --port=8888
```

然后局域网另外机器浏览器访问输出的 URL（通常是 `http://localhost:8888/lab`）。

<http://10.129.242.57:8888/lab>

Q6.字符界面，就是编程自由的意思。

字符界面（CLI, Command Line Interface）确实给编程带来了极大的自由度。在 Linux 下，使用终端可以方便地编写、编译、运行和调试各种语言的代码，比如 Python、C、C++ 等。以下是一些提升开发效率的技巧。

```
[[rocky@jensen code]$ ls -la
total 52
drwxr-xr-x. 2 rocky rocky   70 Feb 19 05:15 .
drwx----- 8 rocky rocky 4096 Feb 19 05:15 ..
-rw-r--r--. 1 rocky rocky   77 Feb 19 05:15 hello.c
-rwxr-xr-x. 1 rocky rocky 17488 Feb 19 05:15 hello_c
-rw-r--r--. 1 rocky rocky   96 Feb 19 05:15 hello.cpp
-rwxr-xr-x. 1 rocky rocky 18096 Feb 19 05:15 hello_cpp
[[rocky@jensen code]$ pwd
/home/rocky/code
[[rocky@jensen code]$ ]
```

C++程序示例，C程序示例

给我一个C++程序示例，一个C程序示例，以及在linux系统下编译、运行的方法。

A. 以下是一个简单的 C++ 和 C 代码示例，并附上在 Linux 下的编译和运行方法。

C++ 示例程序

文件名：`hello.cpp`


```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello, C++!" << std::endl;
5     return 0;
6 }
```

编译 & 运行 (Linux)

```
1 g++ hello.cpp -o hello_cpp
2 ./hello_cpp
```

C 示例程序

文件名: `hello.c`

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello, C!\n");
5     return 0;
6 }
```

编译 & 运行 (Linux)

```
1 gcc hello.c -o hello_c
2 ./hello_c
```

1. `g++` 用于编译 C++ 代码, 而 `gcc` 用于编译 C 代码。
2. `-o` 选项用于指定输出的可执行文件名称。
3. `./hello_cpp` 和 `./hello_c` 运行编译生成的可执行文件。

以下是一些提升开发效率的技巧:

1. Python (脚本语言)

Python 脚本不需要编译, 直接运行:

```
1 python3 script.py # 运行 Python 脚本
```

工具

- `vim` / `nano` / `neovim`: 终端文本编辑器
- `ipython`: 增强版 Python 交互环境

2. C / C++ (编译语言)

在终端编写、编译和运行 C / C++ 程序：

编译 & 运行 C

```
1 gcc program.c -o program
2 ./program
```

编译 & 运行 C++

```
1 g++ program.cpp -o program
2 ./program
```

工具

- `gdb`：GNU 调试器

3. 调试

C / C++ 调试

```
1 g++ -g program.cpp -o program
2 gdb ./program
```

Python 调试

```
1 python3 -m pdb script.py
```

4. 高效开发环境

在 Linux 终端下，可以结合多种工具提升开发体验：

- `tmux` / `screen`：支持多窗口管理
- `vim` / `neovim`：强大的代码编辑器，支持语法高亮
- `cmake`：管理 C/C++ 项目构建
- `lldb`：苹果推荐的调试工具（C++ / C）
- `autopep8` / `black`：Python 代码格式化

5. 一键编译 & 运行（脚本化）

对于 C/C++，可以写一个简单的 `run.sh` 脚本，自动编译和运行：

```
1 #!/bin/bash
2 g++ program.cpp -o program && ./program
```

然后赋予执行权限：

```
1 chmod +x run.sh
2 ./run.sh
```

字符界面让编程更加自由，不受 GUI 约束，适合高效开发和自动化。

2 Linux Shell使用

linux-help (Linux Shell简介) ,

<https://pku.instructuremedia.com/embed/06bda1b0-3342-4705-9c77-e279638f1af2>

学linux命令的小游戏，每次找下一级登录口令

<https://overthewire.org/wargames/bandit/>

定义：在 Linux 或 Unix 系统中，Shell 是一个命令行解释器，它接收用户的命令并将其发送给操作系统内核。

功能：Shell提供字符界面，可以执行程序、编译代码、控制和监测计算机的运行状态。

重要性：大多数底层功能的高效执行需要基于Shell，而不是图形界面（GUI）。因此，Shell在计算机基础学习中不可或缺。使用Shell能够帮助你更好地利用你的设备，提高工作效率。

我们主要讨论的是Linux环境下的Bash，也称为BASH (Bourne Again Shell)。在打开BASH后，你会看到命令提示符，它由多个部分组成：用户名、主机名、工作目录和"\$"符号作为命令提示符。

在这里是rocky。打印当前用户还有一种方法，是whoami。第二部分是主机名，可以理解为计算机的名字，在这里是jensen。

```
1 (base) hfyan@HongfeideMac-Studio ~ % ssh rocky@10.129.242.98
2 Last login: Sat Feb 15 07:50:12 2025 from 162.105.89.132
3 [rocky@jensen ~]$ whoami
4 rocky
5 [rocky@jensen ~]$ pwd
6 /home/rocky
7 [rocky@jensen ~]$
```

可以使用pwd (print working directory) 打印当前工作目录，在这里 /home/rocky 是工作目录，意思就是我们当前处在这个目录中。其他命令还有echo回显，cal是打印今天的日历，clear清屏。

2.1 快捷键和学习资源

使用快捷键来提升效率，快速编辑命令行内容。

Ctrl + U：删除光标前的所有字符。

Ctrl + K：删除光标后的所有字符。

Ctrl + A：定位到命令的首部。

Ctrl + E：定位到命令的尾部。

Ctrl + R：在命令历史记录中进行反向搜索。

快捷键可以减少你输入命令的麻烦。设想这样一种情况，键入了一个非常非常长的指令，但是敲到一半的时候，突然发现整个都错了，需要重新写，按退格键或者一直按着，但是这样子删光整一行，可能需要比较长的时间，可以使用快捷键 `ctrl + u`，删除光标下的所有字符一直到行首。与之相对应的 `ctrl + k`，删除光标下的字符直至行尾。

再比如安装软件如果发现 `permission denied`，因为你不是管理员用户，需要在前面加`sudo`。使用上下键来定位我们之前输入过的命令，然后 `ctrl + a` 定位到命令的首部，插入`sudo`，这时候你就可以直接按`enter`执行了。`ctrl + e` 回到整行的后面。

`trl + r` (reverse search in bash history) 是一个非常重要的快捷键。

`ctrl + l` 与 `clear` 基本等效，但是它前面的东西都清除掉，不会清除当前行输入的命令。

对于学习资源，除了直接在命令后面加上 `--help` 获取简要信息外，还可以使用`man`指令查看详细的用户手册。比如说你不知道`ls`这个指令应该怎么用，可以直接 `ls --help`，更详细的，可以 `man ls`。`man`命令是“manual”的缩写，用于显示命令的手册页 (manual pages)，在这样的这个手册界面中，一般可以使用 `vi` 来定位浏览，`j`向下，`k`向上，如果你要在这个手册中搜索一些信息的话，可以先按正斜杠，然后再输入你要搜索的关键词，比如说`line`，然后按`enter`，这个时候所有的`line`都会被高亮，你再按`n`，也就代表`next`，就可以慢慢的向下搜索了。对于这些手册，可以用`q`来退出它，就是`quit`。

`man`也好，`help`也好，都不是非常的易读，也不是能在短时间内能够看完的，所以你就会想太长不看，有这样一个`too long didn't read`第三方软件。它可以提供简单常用的命令示例，而且只使用一句话来描述。比如说 `tldr ls`，就会看到打印了一些常用的参数组合。比如需要解压缩一个`tar`文件，参数都会比较长，初学可能都记不太住，`tldr tar` 打印一些常见的解压缩以及压缩的指令的用法。

在 Linux 的 Shell 中，可以使用以下方法安装 `tldr` (命令行速查工具)：

方法：使用 `npm` 安装 (推荐)

`npm` 是 Node.js 的包管理器，如果尚未安装，可以先安装 Node.js：

```
1 | sudo dnf install nodejs # Fedora
2 |
```

然后安装 `tldr`：

```
1 | sudo npm install -g tldr
```

另外一个比较有用的命令叫`info`，以咨询一些信息。`info`是什么？叫做GNU Core Utils，是Linux中一些核心的小工具。

先安装

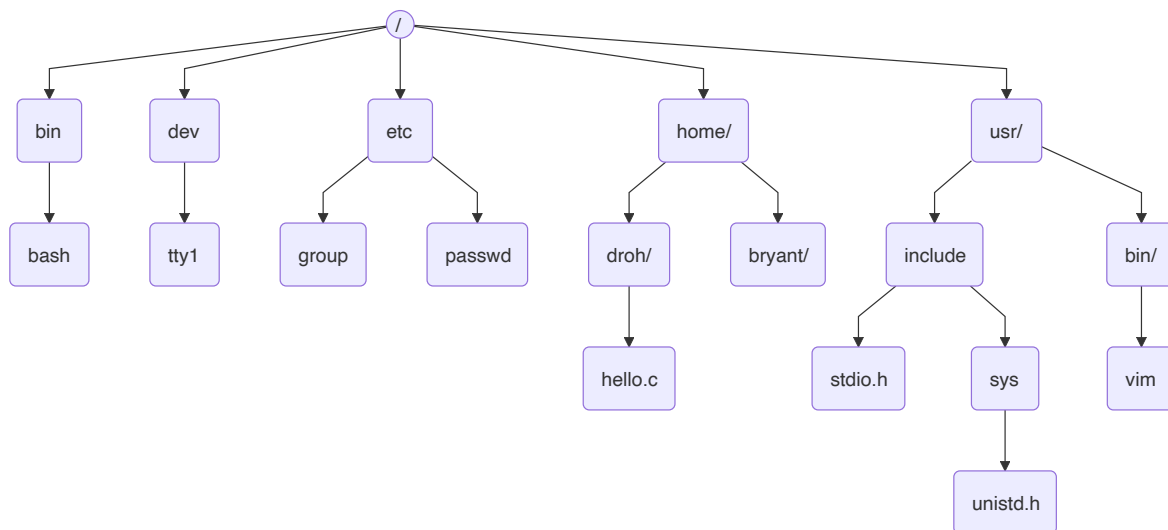
```
1 | sudo npm install -g info
```

如果你运行 `info` 的话，会看到关于这个小工具的一个手册，里面有非常多有意思的工具。如果大家对`shell`的用法感兴趣的话可以探索，在网上也是有`html`版的，当你在这些文档中都找不到好用的信息的时候，可以上网搜索。推荐 <https://unix.stackexchange.com/>，<https://stackoverflow.com/>，比较容易能够获得有效的信息。

2.2 与文件系统交互

在shell中最基础的命令可能是同文件系统进行交互，也就是资源管理器的功能。Linux文件系统一般遵循文件系统层次化标准FHS，在该标准中系统中的一切事物都被视为文件，包括目录，但目录的文件名会有/以示区别，尽管输入目录名称时候，有时候会省略它。

文件以竖形的结构组织在以根目录/为根的文件树中。一般而言，根目录的下一集目录的名字都是固定的。例如所有用户的家目录都存储在home/下。



访问文件系统需要路径的概念，它是指文件树上从某个节点到另一个节点的路径上，所有文件名字符串的连接。从当前工作目录开始的路径称为相对路径，以根目录开始的路径则称为绝对路径。

例如考虑上图中的 `hello.c` 的绝对路径和相对路径，绝对路径是 `/home/droh/hello.c` 而如果我们以 `home` 为工作目录，那么相对路径为 `droh/hello.c`。如果我们在 `droh` 这个用户的家中，则可以直接用 `hello.c` 这个相对路径直接访问它。

有几个特殊路径也是比较常用的，这包括 `/` 根目录和 `~` 家目录以及 `.` 当前目录和 `..` 上一级目录

文件操作

有了路径的概念后，可以开始文件系统的相关操作。在文件系统中浏览和定位主要是用 `ls` 和 `cd` 两个指定

- `ls`：列出目录内容。list directory contents
 - `ls -a`：列出所有文件，包括隐藏文件。隐藏文件一般以 `.` 打头。
 - `ls -l`：以长列表形式列出文件详细信息。
- `cd`：更改工作目录。change directory
 - `cd ..`：返回上一级目录。
 - `cd ~`：返回家目录。
 - `cd -`：返回上一次访问的目录。
- `mkdir`：创建目录。make directory
- `touch`：创建空白文件或修改文件时间戳。

- **rm**：删除文件。remove directory
 - **rm -r**：递归删除目录及其内容。慎重使用，删除后无法通过常规手段恢复。
 - **rm -f**：强制删除，不提示确认。

命令中的通配符，* 匹配任意常的字符串，? 匹配一个字符

例如：rm -rf proxy lab/* 表示删除proxy lab下面的所有文件

rmdir proxy lab/ 删除空目录

- **mv**：移动文件或重命名文件。
- **cp**：复制文件。
 - **cp -r**：递归复制目录及其内容。

文件查看和执行

- **cat**：打印文件内容。concatenate
- **less**：分页查看文件内容。以类似于手册的方式来观察比较长的文件。可以使用手册中的那种键盘操作，按q退出
- **head**、**tail**：查看文件的开头或结尾部分。
- **./**：执行文件。执行文件采用./再加路径的方式。例如：**./bomb**

有关文件的查看，还有wordcount,find,dif等指令。

环境变量

难道不应该使用 **ls** 的完整路径来执行吗？系统使用环境变量中的path变量来完成这件事。所谓环境变量，一般是指在操作系统中用来指定运行环境的一些参数，比如临时文件夹位置，系统文件夹位置等等。

echo \$HOSTNAME 打印主机名，**echo \$PATH** 打印环境变量PATH的值，可以得到一些用冒号分隔的绝对路径。当我们进入一个指令的时候，系统会在这些路径中先顺序搜索可执行文件名，如果找到了就执行。

```
1 [rocky@jensen ~]$ echo $HOSTNAME
2 jensen.instance.cloud.lcpu.dev
3 [rocky@jensen ~]$ echo $PATH
4 /home/rocky/.local/bin:/home/rocky/bin:/usr/local/bin:/usr/bin:/usr/local/sbi
n:/usr/sbin
```

使用export这样的指令来改写环境变量

- **export**：设置环境变量。
- **which**：查找命令的路径。

文件权限

观察 **ls -la** 的输出，由十位组成，第一位表示是否目录，剩下三位为一组，分别代表用户，用户组和其他人对这个文件的权限。文件权限包括读、写、执行，分别用rwx来表示，如果是一个短横线 - 则表示没有这个权限。对于目录来说，执行权限就是搜索，简单的说就是是否能够 **cd** 到这个目录里。

```

1 [rocky@jensen ~]$ ls -la
2 total 28
3 drwx-----. 6 rocky rocky 190 Feb 15 10:02 .
4 drwxr-xr-x. 3 root root 19 Feb 14 04:51 ..
5 -rw-----. 1 rocky rocky 1162 Feb 15 07:50 .bash_history
6 -rw-r--r--. 1 rocky rocky 18 Apr 30 2024 .bash_logout
7 -rw-r--r--. 1 rocky rocky 141 Apr 30 2024 .bash_profile
8 -rw-r--r--. 1 rocky rocky 492 Apr 30 2024 .bashrc
9 -rw-----. 1 rocky rocky 42 Feb 15 09:52 .lessht
10 -rw-r--r--. 1 rocky rocky 483 Feb 14 05:06 login.py
11 drwxr-xr-x. 4 rocky rocky 72 Feb 15 09:55 .npm
12 drwxr-xr-x. 2 rocky rocky 21 Feb 14 06:52 .ollama
13 -rw-----. 1 rocky rocky 7 Feb 14 05:07 .python_history
14 drwx-----. 2 rocky rocky 29 Feb 14 04:51 .ssh
15 drwxr-xr-x. 3 rocky rocky 19 Feb 15 10:02 .tldr
16

```

- **ls -l**：查看文件权限。
- **chmod**：更改文件权限。
 - **chmod u+x**：给用户添加执行权限。即 `chmod u/g/o +/- r/w/x`，表示为用户user，用户group，其他人other来添加+删除-减三种权限

文件打包和压缩

谈到权限我们顺道就可以说到tar这个指令，常会用到tar格式的存档文件，它和zip甚至win rar的区别是它可以保留linux中的文件权限。tar是tape archive的缩写，就是在备份文件的时候常会用到磁带机。用tar打包之后一般会再进行压缩，扩展名为tar.gz指经过gzip算法压缩，而tar.bz2则是经过bzip2算法压缩。

- **tar**：打包文件。
 - **tar czvf**：打包并压缩文件。
 - **tar xzvf**：解压文件。

文件重定向和管道

shell中的程序通常有三个流，也就是输入input stream，输出output stream和错误error stream。一般来说它们都是默认定项到终端中显示的，可以把它们重定向到文件中或者通过管道传输到另外一个程序中，以方便操作和观察。

- **<**：重定向输入到文件。
例如：`$ ./bomb < 0.in` 用0.in这个文件作为bom的输入。
- **>**：重定向输出到文件。
例如：`echo hello > hello.txt` 把echo的输出重定向到hello.text中，这时候 `cat hello.txt` 就看到hello.txt文件中出现了 hello 这样的文字。
- **>>**：追加输出到文件。
例如：`echo hello2 >> hello.txt`
- **2>**：重定向错误输出。

例如: `grep a b 2> log.txt` 在文件b中查找模式a

- `&>`: 可以同时重定向错误和标准输出。

例如: `grep a b &> log.txt`

- `|`: 管道, 将前一个命令的输出作为后一个命令的输入。

例如: `./bomb < phase | grep solution` 从输出中打印那些恰好含有solution这个词的那些行

Shell脚本示例重定向 01017:装箱问题

<http://cs101.openjudge.cn/practice/01017>

提交代码Wrong Answer, 借助测试数据找到哪里出错。

状态: **Wrong Answer**

源代码

```
import math
determine_2with3=[0,5,3,1]

while True:
    n1,n2,n3,n4,n5,n6 = map(int,input().split())
    if n1+n2+n3+n4+n5+n6==0:
        break
    boxes = n4+n5+n6+math.ceil(n3/4)
    spaceleft_for_b = n4*5 + determine_2with3[n3%4]
    if n2 > spaceleft_for_b:
        boxes += math.ceil(n2-spaceleft_for_b/9)
    spaceleft_for_a = (boxes-n6)*36-n5*25-n4*16-n3*9-n2*4
    if n1 > spaceleft_for_a:
        boxes += math.ceil(n1-spaceleft_for_a/36)
    print(boxes)
```

基本信息

#: 48302350
题目: 01017
提交人: HFYan(GMyhf)
内存: 3660kB
时间: 42ms
语言: Python3
提交时间: 2025-02-18 13:57:02

进到测试数据目录

```
1 [rocky@jensen 1017]$ ls -l
2 total 92
3 -rw-r--r--. 1 rocky rocky 31873 Oct  1  2023 aamy.out
4 -rw-r--r--. 1 rocky rocky  586 Nov 26  2008 d.c
5 -rw-r--r--. 1 rocky rocky 34950 Nov 26  2008 d.in
6 -rw-r--r--. 1 rocky rocky  7805 Nov 26  2008 d.out
7 -rw-r--r--. 1 rocky rocky  465 Oct 11 08:49 e1.py
8 -rw-r--r--. 1 rocky rocky  217 Dec  6  2008 me.cpp
9 -rw-r--r--. 1 rocky rocky  1355 Oct 11 09:24 testing_code.py
10 [rocky@jensen 1017]$ ls ../../offlinejudge.zsh
```

其中e1.py保存的是WA的程序

```
1 import math
2 determine_2with3=[0,5,3,1]
3
4 while True:
5     n1,n2,n3,n4,n5,n6 = map(int,input().split())
6     if n1+n2+n3+n4+n5+n6==0:
7         break
```



```

8     boxes = n4+n5+n6+math.ceil(n3/4)
9     spaceleft_for_b = n4*5 + determine_2with3[n3%4]
10    if n2 > spaceleft_for_b:
11        boxes += math.ceil(n2-spaceleft_for_b/9)
12    spaceleft_for_a = (boxes-n6)*36-n5*25-n4*16-n3*9-n2*4
13    if n1 > spaceleft_for_a:
14        boxes += math.ceil(n1-spaceleft_for_a/36)
15    print(boxes)
16

```

offlinejudge.zsh 是Shell脚本

```

1  cd $2
2  for i in *.in; do
3      diff -y <(python3 "$1" < "$i") "${i%.*}.out"
4  done

```

这段 Bash 脚本用于批量对比 Python 脚本的输出与预期输出。以下是其工作原理的详细解析：

代码解析

```
1  cd $2
```

- 进入用户提供的目录 `$2`（脚本的第二个参数）。

```
1  for i in *.in; do
```

- 遍历当前目录下所有以 `.in` 结尾的文件（即输入文件）。

```
1  diff -y <(python3 "$1" < "$i") "${i%.*}.out"
```

- `"$1"` 是脚本的第一个参数，表示 Python 脚本的路径。
- `python3 "$1" < "$i"`：运行该 Python 脚本，并使用 `$i` 作为输入文件（`*.in`）。
- `<(...)` 是 **进程替换**，将 Python 脚本的输出作为 `diff` 命令的一个输入。
- `"${i%.*}.out"` 解析如下：
 - `"${i%.*}"` 移除 `$i` 文件名的最后一个 `.` 及其后缀，即 `input.in` → `input`。
 - `"${i%.*}.out"` 生成相应的 `.out` 文件名（如 `input.out`）。
- `diff -y`：
 - `diff` 比较两个文件的内容。
 - `-y` 以 **并排（side-by-side）** 方式显示差异，方便人工对比。

使用示例

假设目录结构如下：

```
1 | 1017/  
2 | — aamy.out  
3 | — d.c  
4 | — d.in  
5 | — d.out  
6 | — e1.py  
7 | — me.cpp  
8 | — testing_code.py
```

执行：

```
1 | bash ../../offlinejudge.zsh e1.py ./
```

脚本会：

1. 进入 `./` 目录。
2. 遍历所有 `.in` 文件：
 - 对 `d.in`，运行 `python3 e1.py < d.in`，并将其结果与 `d.out` 进行 `diff -y` 对比。

总结

此脚本适用于测试 **Python** 脚本的标准输入/输出是否符合预期，特别是在编程竞赛、自动化测试或算法开发中。

```
hfyang — rocky@jensen:~/data/20071224
[rocky@jensen 1017]$ which sh
/usr/bin/sh
[rocky@jensen 1017]$ ls -l /usr/bin/sh
lrwxrwxrwx. 1 root root 4 Apr 30 2024 /usr/bin/sh -> bash
[rocky@jensen 1017]$ bash ../../offlinejudge.zsh e1.py ./
86 86
231 231
137 137
115 115
219 219
192 | 116
142 | 142
148 148
198 198
218 218
269 | 186
158 | 158
90 90
160 160
227 227
125 125
162 162
176 176
113 113
126 126
237 237
192 192
183 183
184 184
130 130
244 244
199 199
195 195
142 142
178 | 119
213 213
238 238
269 269
162 162
151 151
179 179
```

Python脚本管道示例27300:模型整理

<http://cs101.openjudge.cn/practice/27300/>

Wrong Answer

状态: **Wrong Answer**

源代码

```
# -*- coding: utf-8 -*-
"""
Created on Fri Feb 14 19:58:10 2025

@author: Liren Chen
"""

n = int(input())
mx, sj = [], {}
for i in range(0, n):
    a, b = input().split('-')
    if a not in sj:
        sj[a] = [[], []]
        mx.append(a)
        if b[-1] == 'B':
            if '.' in b:
                b = float(b[0:-1])
            else:
                b = int(b[0:-1])
            sj[a][1].append(b)
        else:
            if '.' in b:
                b = float(b[0:-1])
            else:
                b = int(b[0:-1])
            sj[a][0].append(b)
    else:
        if b[-1] == 'B':
            if '.' in b:
                b = float(b[0:-1])
            else:
                b = int(b[0:-1])
            sj[a][1].append(b)
        else:
            if '.' in b:
                b = float(b[0:-1])
            else:
                b = int(b[0:-1])
            sj[a][0].append(b)
mx.sort()
for i in mx:
    sj[i][0].sort()
    sj[i][1].sort()
    if len(sj[i][0]) == 0:
        print(i + ': ', '.join(str(j) + 'B' for j in sj[i][1]))
    if len(sj[i][1]) == 0:
        print(i + ': ', '.join(str(j) + 'M' for j in sj[i][0]))
    else:
        print(i + ': ', '.join(str(j) + 'M' for j in sj[i][0]) + ', ' + '.join(str(j) + 'B' for j in sj[i][1]))
```

基本信息

#: 48286780
题目: 27300
提交人: HfYan(GMyhf)
内存: 3724kB
时间: 27ms
语言: Python3
提交时间: 2025-02-14 21:08:14

e1.py

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Fri Feb 14 19:58:10 2025
4
5 @author: Liren Chen
6 """
7
8 n = int(input())
9 mx, sj = [], {}
10 for i in range(0, n):
11     a, b = input().split('-')
12     if a not in sj:
13         sj[a] = [[], []]
14         mx.append(a)
15         if b[-1] == 'B':
```

```

16         if '.' in b:
17             b = float(b[0:-1])
18         else:
19             b = int(b[0:-1])
20             sj[a][1].append(b)
21     else:
22         if '.' in b:
23             b = float(b[0:-1])
24         else:
25             b = int(b[0:-1])
26             sj[a][0].append(b)
27     else:
28         if b[-1]=='B':
29             if '.' in b:
30                 b = float(b[0:-1])
31             else:
32                 b = int(b[0:-1])
33             sj[a][1].append(b)
34         else:
35             if '.' in b:
36                 b = float(b[0:-1])
37             else:
38                 b = int(b[0:-1])
39             sj[a][0].append(b)
40     mx.sort()
41     for i in mx:
42         sj[i][0].sort()
43         sj[i][1].sort()
44         if len(sj[i][0])==0:
45             print(i+' : '+' , '.join(str(j)+'B' for j in sj[i][1]))
46         if len(sj[i][1])==0:
47             print(i+' : '+' , '.join(str(j)+'M' for j in sj[i][0]))
48         else:
49             print(i+' : '+' , '.join(str(j)+'M' for j in sj[i][0])+' , '+' , '.join(str(j)+'B' for j in sj[i][1]))
50

```

```

1 [rocky@jensen 27300]$ ls
2 0.in  10.in  11.in  12.in  13.in  14.in  15.in  16.in  17.in  18.in
   19.in  1.in  2.in  3.in  4.in  5.in  6.in  7.in  8.in  9.in
   e1.py
3 0.out 10.out 11.out 12.out 13.out 14.out 15.out 16.out 17.out 18.out
   19.out 1.out 2.out 3.out 4.out 5.out 6.out 7.out 8.out 9.out

```

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Fri Feb 14 19:58:10 2025

@author: Liren Chen
"""

n = int(input())
mx, sj = [], {}
for i in range(0, n):
    a, b = input().split('-')
    if a not in sj:
        sj[a] = [], []
        mx.append(a)
        if b[-1] == 'B':
            if '.' in b:
                b = float(b[0:-1])
            else:
                b = int(b[0:-1])
            sj[a][1].append(b)
        else:
            if '.' in b:
                b = float(b[0:-1])
            else:
                b = int(b[0:-1])
            sj[a][0].append(b)
    else:
        if b[-1] == 'B':
            if '.' in b:
                b = float(b[0:-1])
            else:
                b = int(b[0:-1])
            sj[a][1].append(b)
        else:
            if '.' in b:
                b = float(b[0:-1])
            else:
                b = int(b[0:-1])
            sj[a][0].append(b)
mx.sort()
"e1.py" 50L, 1243B

```

testing_code.py

https://github.com/GMyhf/2024fall-cs101/blob/main/code/testing_code.py

```

1  # ZHANG Yuxuan
2  import subprocess
3  import difflib
4  import os
5  import sys
6
7  def test_code(script_path, infile, outfile):
8      command = ["python", script_path] # 使用Python解释器运行脚本
9      with open(infile, 'r') as fin, open(outfile, 'r') as fout:
10         expected_output = fout.read().strip()
11         # 启动一个新的子进程来运行指定的命令
12         process = subprocess.Popen(command, stdin=fin,
13                                     stdout=subprocess.PIPE)
14         actual_output, _ = process.communicate()
15         if actual_output.decode().strip() == expected_output:
16             return True

```

```

16         else:
17             print(f"Output differs for {infile}:")
18             diff = difflib.unified_diff(
19                 expected_output.splitlines(),
20                 actual_output.decode().splitlines(),
21                 fromfile='Expected', tofile='Actual', lineterm=''
22             )
23             print('\n'.join(diff))
24             return False
25
26
27 if __name__ == "__main__":
28     # 检查命令行参数的数量
29     if len(sys.argv) != 2:
30         print("Usage: python testing_code.py <filename>")
31         sys.exit(1)
32
33     # 获取文件名
34     script_path = sys.argv[1]
35
36     #script_path = "class.py" # 你的Python脚本路径
37     #test_cases = ["d.in"] # 输入文件列表
38     #expected_outputs = ["d.out"] # 预期输出文件列表
39     # 获取当前目录下的所有文件
40     files = os.listdir('.')
41
42     # 筛选出 .in 和 .out 文件
43     test_cases = [f for f in files if f.endswith('.in')]
44     test_cases = sorted(test_cases, key=lambda x: int(x.split('.')[0]))
45     #print(test_cases)
46     expected_outputs = [f for f in files if f.endswith('.out')]
47     expected_outputs = sorted(expected_outputs, key=lambda x:
48 int(x.split('.')[0]))
49     #print(expected_outputs)
50
51     for infile, outfile in zip(test_cases, expected_outputs):
52         if not test_code(script_path, infile, outfile):
53             break

```

这个程序实际上使用了管道 (pipe) 来捕获 Python 子进程的输出

代码功能：

此 Python 脚本的主要目的是自动测试另一个 Python 程序的输出是否与预期结果一致，适用于多组测试用例的批量验证。

执行流程解析：

① 导入必要模块

```
1 import subprocess # 用于启动子进程, 执行 Python 程序
2 import difflib     # 生成输出的差异比较
3 import os          # 文件和目录操作
4 import sys         # 处理命令行参数
```

② 核心函数: `test_code()`

```
1 def test_code(script_path, infile, outfile):
```

该函数用于执行目标 Python 脚本, 并将其输出与预期结果进行比对。

(1) 构建 Python 命令

```
1 command = ["python", script_path]
```

- 通过 `subprocess` 模块构造执行命令, `script_path` 是目标 Python 脚本路径。
- 假设输入为 `python class.py`, 此处 `script_path` 指的是 `"class.py"`。

(2) 读取输入和预期输出

```
1 with open(infile, 'r') as fin, open(outfile, 'r') as fout:
2     expected_output = fout.read().strip()
```

- 打开输入文件 `infile` 和输出文件 `outfile`。
- `expected_output` 保存了去除首尾空白字符的预期结果。

(3) 运行目标脚本

```
1 process = subprocess.Popen(command, stdin=fin, stdout=subprocess.PIPE)
2 actual_output, _ = process.communicate()
```

- `subprocess.Popen()` 启动一个子进程, 执行目标 Python 脚本。
 - `stdin=fin`: 将输入文件作为 Python 程序的输入。
 - `stdout=subprocess.PIPE`: 捕获程序的输出, 方便与预期结果比较。
- `process.communicate()`: 等待子进程完成, 获取输出结果。

(4) 输出对比

```
1 if actual_output.decode().strip() == expected_output:
2     return True
```

- 将子进程输出解码为字符串并去除空白字符, 若与预期输出一致, 返回 `True`, 表示该用例通过。

(5) 生成差异报告

如果输出不一致, 则输出差异信息:


```

1 print(f"Output differs for {infile}:")
2 diff = difflib.unified_diff(
3     expected_output.splitlines(),
4     actual_output.decode().splitlines(),
5     fromfile='Expected', tofile='Actual', lineterm=''
6 )
7 print('\n'.join(diff))
8 return False

```

- `difflib.unified_diff()`: 生成统一格式的输出差异。
 - `expected_output.splitlines()`: 将期望输出按行分割。
 - `actual_output.decode().splitlines()`: 将实际输出按行分割。
 - `fromfile='Expected'` 和 `tofile='Actual'`: 差异报告的标签。
 - `lineterm=''`: 避免多余的换行符。
- 通过 `print()` 打印出详细的输出差异信息。

③ 主程序入口

```

1 if __name__ == "__main__":

```

确保以下代码仅在脚本被直接运行时执行。

(1) 检查参数

```

1 if len(sys.argv) != 2:
2     print("Usage: python testing_code.py <filename>")
3     sys.exit(1)

```

- `sys.argv` 读取命令行参数。
- 如果没有提供目标 Python 脚本路径，程序会提示正确用法并退出。

(2) 获取输入和输出文件

```

1 files = os.listdir('.')

```

- `os.listdir()` 获取当前目录下的所有文件列表。

筛选 `.in` 和 `.out` 文件:

```

1 test_cases = [f for f in files if f.endswith('.in')]
2 expected_outputs = [f for f in files if f.endswith('.out')]

```

- 过滤出以 `.in` 结尾的输入文件和 `.out` 结尾的输出文件。

按数字顺序排序:

```

1 test_cases = sorted(test_cases, key=lambda x: int(x.split('.')[0]))
2 expected_outputs = sorted(expected_outputs, key=lambda x:
    int(x.split('.')[0]))

```

- 按文件名前缀的数字部分排序，确保输入和输出文件一一对应。
 - 例如：1.in 对应 1.out，2.in 对应 2.out。

(3) 执行测试

```

1 for infile, outfile in zip(test_cases, expected_outputs):
2     if not test_code(script_path, infile, outfile):
3         break

```

- `zip()` 将输入和输出文件配对遍历。
- 对每个输入输出文件调用 `test_code()`，若输出不匹配，立即终止测试（通过 `break` 退出循环）。

④ 执行示例

1. 运行命令：

```

1 python ../../testing_code.py e1.py

```

2. 输出情况：

- 若所有测试用例均通过，无输出。
- 若有差异，例如 0.in 失败，输出类似以下结果：

```

1 Traceback (most recent call last):
2   File "/mnt/data/20071224-POJTestData/4000_/27300/e1.py", line 23, in
    <module>
3     b = float(b[0:-1])
4 ValueError: could not convert string to float: '1.3B'
5 Output differs for 0.in:
6 --- Expected
7 +++ Actual
8 @@ -1,2 +0,0 @@
9 -Bert: 110M, 340M
10 -GPT: 350M, 1.3B, 175B
11

```

总结

该脚本自动化测试 Python 程序的输出，具有以下特点：

1. 批量测试：对所有 `.in` 文件进行测试并与相应的 `.out` 文件比对。
2. 差异报告：使用 `diff` 输出详细差异，方便调试。
3. 自动终止：若发现错误，立即终止后续测试，节省时间。

4. 兼容性强：适用于任何标准输入/输出形式的 Python 程序测试。

```
[rocky@jensen 27300]$ python ../../testing_code.py e1.py
Traceback (most recent call last):
  File "/mnt/data/20071224-POJTestData/4000_/27300/e1.py", line 23, in <module>
    b = float(b[0:-1])
ValueError: could not convert string to float: '1.3B'
Output differs for 0.in:
--- Expected
+++ Actual
@@ -1,2 +0,0 @@
-Bert: 110M, 340M
-GPT: 350M, 1.3B, 175B
[rocky@jensen 27300]$
```

我们介绍了Shell的基本概念、常用命令、快捷键、文件操作、环境变量、文件权限、打包压缩以及重定向和管道。

3 本地大语言模型安装和测试

在本地环境部署大语言模型（LLM）并进行测试。可以选择使用图形界面工具如 <https://lmstudio.ai> 成部署工作。

测试内容包括选择若干编程题目，确保这些题目能够在所部署的LLM上得到正确解答，并通过所有相关的测试用例（即状态为Accepted）。选题应来源于在线判题平台，例如 OpenJudge、Codeforces、LeetCode 或洛谷等，同时需注意避免与已找到的AI接受题目重复。已有的AI接受题目列表可参考以下链接：

https://github.com/GMyhf/2025spring-cs201/blob/main/AI_accepted_locally.md

3.1 LLM 的基本概念

在学习 LLM 之前，需要掌握一些基础概念：

3.1.1 Transformer 结构

Transformer 是 LLM 的核心架构，由 Google 研究团队在 2017 年提出，核心机制包括：

- 自注意力机制（Self-Attention）：使模型能关注输入序列中的重要部分，提高文本理解能力。
- 多头注意力（Multi-Head Attention）：增强模型捕捉不同模式的能力。
- 前馈网络（Feedforward Network, FFN）：提高模型表达能力。

LLM（如 GPT 系列、LLaMA、PaLM、DeepSeek）都是基于 Transformer 发展而来。

3.1.2 预训练与微调

- 预训练（Pre-training）：在大规模数据上训练模型，使其具备基础的语言理解能力。
- 微调（Fine-tuning）：针对特定任务优化模型，如医学 NLP、代码生成等。

3.1.3 GPU 在 LLM 训练中的作用

- **并行计算加速**：LLM 需要处理海量数据，GPU 的并行计算能力可提高训练效率。
- **模型推理优化**：部署 LLM 需要高效推理，GPU（如 NVIDIA A100、H100）在 AI 服务器中发挥重要作用。

OpenAI o3斩获IOI金牌冲榜全球TOP 18，自学碾压顶尖程序员！48页技术报告公布，<https://mp.weixin.qq.com/s/rHzZqTBhLBrb-FPtHhEYug>

Competitive Programming with Large Reasoning Models, <https://arxiv.org/pdf/2502.06807>

OpenAI o3却在无人启发的情况下，通过强化学习中自己摸索出了一些技巧。即o3在推理过程中展现出更具洞察力和深度思考的思维链。对于验证过程较为复杂的问题，o3会采用一种独特的策略：**先编写简单的暴力解法，牺牲一定效率来确保正确性，然后将暴力解法的输出与更优化的算法实现进行交叉检查。**

```
def test_random_small():
    import random
    random.seed(1)
    for n in range(1,5):
        for m in range(1,n+1):
            s = ''.join(random.choice('ab') for _ in range(n))
            labels = solve_bruteforce_given_s(s,m)
            for k in [1, len(labels)//2+1, len(labels)]:
                k = max(1, min(k, len(labels)))
                ans = solve_main(s,m,k)
                brute_ans = labels[k-1]
                if ans != brute_ans:
                    print("Mismatch on s:",s,"n",n,"m",m,"k",k,"expected",brute_ans,"got",ans)
                    return False
            print("random small tests passed")
    return True

test_random_small()
```

random small tests passed

公众号 · 新智元

3.2 机器是Mac Studio（看本地机器配置和性能）

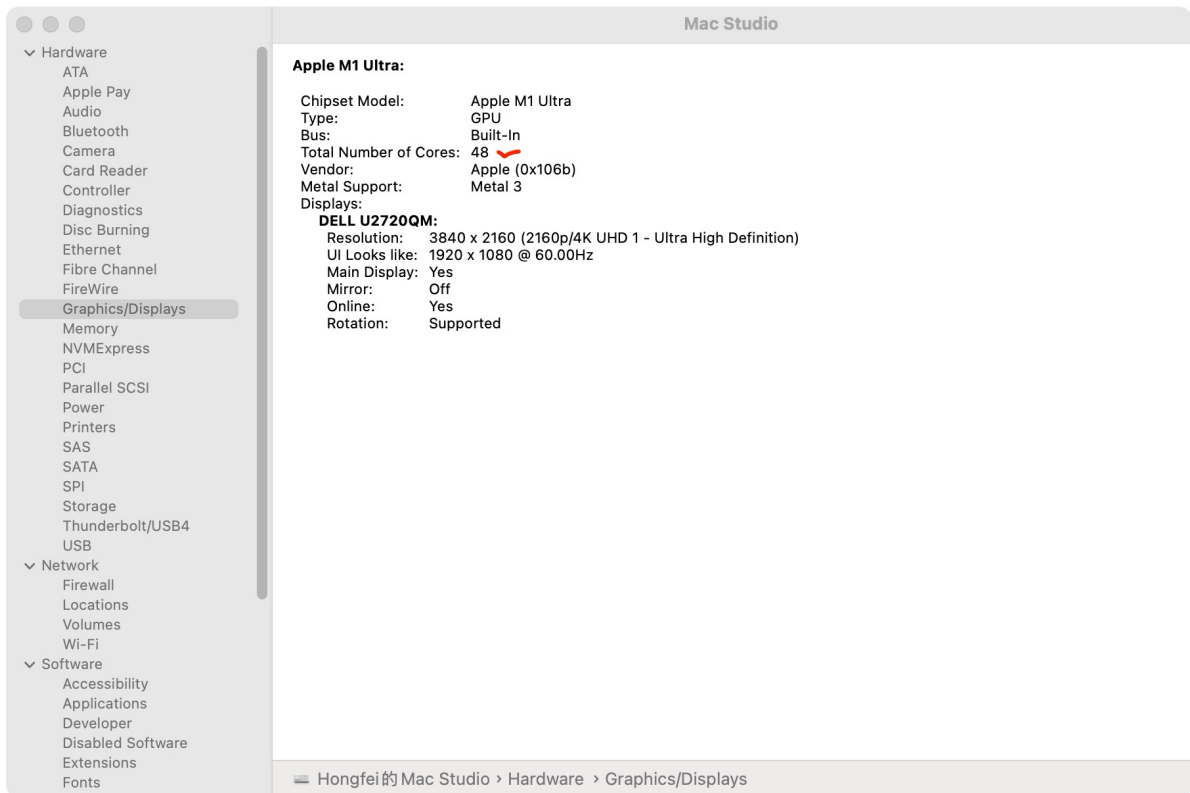


Apple M1 Ultra 是 Apple 芯片系列中的一员，专为高性能需求设计，特别是在 Mac Studio 等设备中使用。M1 Ultra 的配置包括了中央处理器（CPU）、图形处理器（GPU）以及统一内存架构（Unified Memory Architecture, UMA），其中统一内存可供 CPU、GPU 以及其他组件共享。

关于 GPU 内存

在 M1 Ultra 中，64GB 内存实际上是整个系统共享的统一内存容量，这意味着这64GB内存是由CPU、GPU及其他组件共同使用的，而不是专门分配给GPU的独立内存。这种设计极大地提高了灵活性和性能表现，尤其是在处理复杂图形任务或多任务处理场景下。

- **统一内存架构：**Apple 的设计理念是通过统一内存架构来提升性能和效率。这种架构允许 GPU 和 CPU 访问相同的内存池，减少了数据复制的需求，并且可以更灵活地根据需要分配内存资源。
- **M1 Ultra 的 GPU 资源：**M1 Ultra 配备了一个强大的 48 核心 GPU。尽管没有“专用”的 GPU 显存，但其可以从整个 64GB 统一内存中获取所需的工作内存。这对于许多图形密集型应用来说是非常有利的，因为它避免了传统显存与主存之间可能存在的瓶颈。



Geekbench AI测试, <https://browser.geekbench.com>

3.3 本地机器安装LM Studio及测试

下载 LM-Studio-0.3.25-2-arm64, 531MB。

Q1. 装好了 LM Studio, 但是无模型

模型下载有问题, 模型列表都加载不出来?

A: 在app settings下面有个代理选项。command+逗号, 在一堆设置下面。

代理开了。点左侧搜索那个按钮, 就有很多模型列出来了

Q2. MLX 与 GGUF 的主要区别

MLX和GGUF都是在处理大型语言模型 (LLMs) 领域中使用的文件格式, 它们各自有不同的设计目标和应用场景。以下是它们的主要区别:

MLX 格式

MLX 是由 Hugging Face 开发的一种二进制模型格式，旨在提供高效的模型存储和加载机制。它旨在替代传统的 PyTorch `.pt` 或 TensorFlow `.pb` 格式，解决这些格式在加载速度和存储效率上的不足。

主要特点：

1. 高效的序列化：
 - MLX 使用高效的二进制序列化方法，能够显著减少模型文件的大小，从而节省存储空间并加快下载速度。
2. 优化的加载性能：
 - 由于采用了二进制格式，MLX 在模型加载时比文本格式（如 `.pt`）更快，这对于需要频繁加载模型的应用场景尤为重要。
3. 跨平台兼容性：
 - MLX 设计为跨平台兼容，可以在不同的操作系统和硬件架构上高效地加载和运行模型。
4. 支持多种框架：
 - 虽然 MLX 最初由 Hugging Face 开发，但它旨在支持多种深度学习框架，使得不同框架训练的模型可以统一使用 MLX 格式。

应用场景：

- 部署在服务器或云端的模型服务。
- 需要快速加载和推理的应用，如实时聊天机器人、自动翻译等。

GGUF 格式

GGUF (GPT-Generative Universal Format) 是一种专为大型语言模型设计的通用文件格式，旨在提供更高的灵活性和可扩展性。它最初由 Stability AI 开发，用于其 StableLM 系列模型。

主要特点：

1. 模块化和可扩展性：
 - GGUF 允许模型开发者定义自定义层和参数，使其能够适应各种复杂的模型架构。
2. 支持多模型集成：
 - GGUF 支持在一个文件中包含多个模型或模型的不同部分，便于管理和部署复杂的模型系统。
3. 丰富的元数据支持：
 - GGUF 文件可以包含丰富的元数据信息，如模型训练配置、超参数、版本信息等，有助于模型的管理和调试。
4. 优化的存储结构：
 - 通过优化的数据存储结构，GGUF 在保持高灵活性的同时，也尽量减少了文件大小和加载时间。

应用场景：

- 开发和部署具有复杂架构的自定义大型语言模型。
- 需要集成多个模型或模型组件的应用场景。
- 需要详细管理和跟踪模型元数据的项目。

MLX 与 GGUF 的主要区别

特性	MLX	GGUF
设计目标	提供高效的序列化和加载性能	提供高度的模块化、可扩展性和丰富的元数据支持
灵活性	较高的序列化效率，但灵活性相对较低	高度灵活，支持自定义层和多模型集成
应用场景	主要用于高效部署和推理	主要用于开发复杂、可扩展的大型语言模型
开发者	Hugging Face	Stability AI
社区和支持	广泛支持，集成于 Hugging Face 生态系统	主要在 Stability AI 及其生态系统中使用

选择建议

- 如果你需要高效的模型序列化和快速的加载性能，并且你的模型架构相对标准，**MLX** 是一个更好的选择。它特别适合用于部署和生产环境中的模型服务。
- 如果你正在开发具有复杂架构的自定义大型语言模型，或者需要在一个文件中集成多个模型组件，并且需要详细的元数据管理，**GGUF** 将更适合你的需求。它提供了更高的灵活性和扩展性，适用于研究和开发阶段。

总结

MLX 和 GGUF 各有其独特的优势和应用场景。MLX 更侧重于高效的序列化和加载性能，适用于生产环境中的模型部署；而 GGUF 则提供了更高的灵活性和丰富的元数据支持，适合开发和集成复杂的自定义模型。根据你的具体需求选择合适的格式，可以更好地满足项目的需求。

2025/2/15: 我在下载 lmstudio-community/DeepSeek-R1-Distill-Qwen-32B-GGUF/DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf。网速只有1.24MB/s，还需要03:17:00 left。

Q3. Distill,32B,Q4这些参数是什么意思？

DeepSeek-R1-Distill-Qwen-32B-GGUF/DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf

A: 问题涉及 **GGUF** 格式的量化模型文件，特别是 `DeepSeek-R1-Distill-Qwen-32B-GGUF/DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf` 这类文件名的参数含义。以下是详细解析：

文件名结构解析

`DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf`

1. **DeepSeek-R1-Distill**

- DeepSeek-R1-Distill**：指的是 DeepSeek 实验室训练的第一代（R1）蒸馏版（Distill）模

型。

- **蒸馏 (Distill)** 通常意味着这个模型是从一个更大的模型（如 DeepSeek 原始模型）压缩或优化而来的，以提高推理速度并减少计算需求。

2. Qwen-32B

- **Qwen**: 表示这个模型基于 Qwen（通义千问）架构，由阿里巴巴 DAMO 研发。
- **32B**: 表示该模型有 **320 亿 (32 Billion)** 参数，是一个较大的模型。

3. GGUF

- **GGUF (GPT-Generated Unified Format)** 是一种 **高效的 LLM 量化格式**，专为 GGML/GGUF 兼容的推理引擎（如 llama.cpp）设计。
- 这种格式可以优化大语言模型在 CPU 和 GPU 上的推理速度。

4. Q4_K_M

- **Q4**: 指的是 **量化精度为 4-bit**（表示每个权重只占 4 比特）。
- **K**: 代表 **K-quants 量化方案**，用于更好的推理精度和平衡计算效率。
- **M**: 通常表示 **Mixed**（混合量化），意味着模型可能使用了混合精度量化（例如部分权重 4-bit，部分 8-bit）。

Q4_K_M 代表的量化方案

- **Q4** 指 4-bit 量化，意味着模型的参数被压缩为 4-bit，而不是原始的 16-bit 或 32-bit 浮点数。

在模型压缩中，量化通常是指从浮点数到整数的映射过程，目前比较常用的是 8 比特整数量化，即 INT8 量化。针对神经网络模型，通常有两种类型的数据需要进行量化，分别是权重（也称为模型参数）和激活，它们都以浮点数形式进行表示和存储。量化过程可以看作一个函数变换，目标是将连续的输入数值映射为离散的输出集合。

- **K** (K-quant) 是一种增强的量化技术，可以提高量化模型的精度，减少推理误差。
- **M** (Mixed) 表示 **混合量化**，通常意味着：
 - 可能对不同层使用不同的量化精度。
 - 可能混合不同的块 (blockwise quantization)。

GGUF 量化格式的选择

你可能会看到类似的 GGUF 文件，例如：

- **Q4_K_S** (4-bit K-quants, Small)
- **Q5_K_M** (5-bit K-quants, Mixed)
- **Q8_K** (8-bit K-quants)

通常：

- **Q4_K_M**: 平衡性能和速度，适合中等计算资源（如 CPU + 高端 GPU）。
- **Q5_K_M**: 更好的精度，但计算开销更大。
- **Q8_K**: 最高精度，但需要更多显存 (VRAM)。

如何使用这些模型

如果你想在 **CPU 或 GPU** 上运行这个 GGUF 量化模型，可以使用 `llama.cpp` 或 `ollama`：

使用 llama.cpp

```
1 | ./main -m DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf -p "Hello, how are you?"
```

使用 Ollama

```
1 | ollama run qwen:DeepSeek-R1-Distill-Qwen-32B-Q4_K_M
```

总结

- **DeepSeek-R1-Distill** → 代表 DeepSeek 训练的 R1 版本的蒸馏（优化）模型。
- **Qwen-32B** → 基于 Qwen（通义千问）架构，**32B** 表示 320 亿参数。
- **GGUF** → 现代优化的量化模型格式。
- **Q4_K_M** → **4-bit** 量化，**K-quants** 技术，**混合量化（Mixed）**，适合 CPU 运行，也能在低端 GPU 上执行。

如果你需要运行这个模型，可以告诉我你的 **硬件配置**，我可以提供更合适的建议！

Q4.根据我的硬件配置，请提供更合适的建议

Mac Studio: Processor Apple M1 Ultra @ 3.21 GHz, 20 核，48核GPU，64GB 内存。统一内存架构 (UMA) 让 CPU 和 GPU 共享同一内存吃，无需数据复制。OS macOS 15.3 (Build 24D60), AI Framework: Core ML, AI Backend: GPU.

A: 你的 Mac Studio (M1 Ultra) 拥有强大的计算能力，尤其是 **统一内存架构 (UMA)** 可以让 CPU 和 GPU 高效共享内存。这使得在 **Mac 上运行 GGUF 量化模型** 时，可以利用 GPU 加速推理。

最佳方案：使用 llama.cpp + Metal 后端

`llama.cpp` 已经支持 **Apple Metal API**，能高效利用 M1/M2 GPU 进行推理，适合 GGUF 格式的 DeepSeek 和 Qwen 量化模型。

1. 安装 `llama.cpp` (带 Metal 支持)

```
1 | brew install llama.cpp
```

2. 运行模型

下载 **DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf** 后，在终端运行：

```
1 | llama-cli -m DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf --n-gpu-layers 100
```

/Users/hfyan/.lmstudio/models/lmstudio-community/DeepSeek-R1-Distill-Qwen-32B-GGUF

参数解析：

- `--n-gpu-layers 100`：将 100 层放到 GPU（M1 Ultra 的 GPU 和 RAM 共享内存，适合大模型）。
- `-m`：指定 GGUF 模型文件。

3. 如果 `brew` 版本不支持 Metal，加载 Metal 后端

```
1 git clone https://github.com/ggerganov/llama.cpp
2 cd llama.cpp
3 make LLAMA_METAL=1
4 ./main -m DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf --n-gpu-layers 100
```

替代方案：使用 `Ollama`

`Ollama` 是 Mac 原生 AI 推理框架，内置 Metal 加速，推荐用于 简化运行流程。

1. 安装 `Ollama`

```
1 brew install ollama
```

2. 运行 DeepSeek Qwen

```
1 ollama run qwen:32b
```

或者直接加载本地 GGUF 模型：

```
1 ollama create qwen-32b -m DeepSeek-R1-Distill-Qwen-32B-Q4_K_M.gguf
2 ollama run qwen-32b
```

总结

方式	优势	适用场景
<code>llama.cpp</code>	轻量级、Metal 支持	终端用户，自定义推理参数
<code>Ollama</code>	简单易用，自动优化	一键运行 AI 模型

推荐方案：

- 如果你喜欢终端控制和调优，用 `llama.cpp`（带 Metal）。
- 如果想快速运行，用 `Ollama`。

你的 M1 Ultra 性能足够运行 **Qwen-32B** 量化版，如果你想尝试更大的模型（如 Qwen-72B），可以用 **Q5_K_M** 或 **Q8_K** 量化版本，但需要更多 RAM。

如果你想要更好的推理性能优化，我可以帮你调整 **Metal** 线程数、缓存策略 等参数！

装好后，图像界面的本地LLM。

可以本地做编程题目。

再比如：

3.4 LM Studio API 交互

目前，我正在 macOS 机器上使用 **LM Studio 0.3.25** 版本，加载的大模型为 **Qwen3-32B**（GGUF 格式，上下文长度 4096），模型文件大小约为 **18.55GB**。

3.4.1 图形界面交互

通过 LM Studio 的图形化界面，可以便捷地进行自然语言交互，例如查询编程相关问题。用户只需在对话框中输入问题，模型即可实时生成解答，体验类似于使用 ChatGPT。例如：

在图形界面的 LM Studio 中可以查询编程问题，如：

给出python:

在一个有180人的大班级中，存在两个人生日相同的概率非常大，现给出每个学生的学号，出生月日。试找出所有生日相同的学生。

输入

第一行为整数n，表示有n个学生， $n < 100$ 。

此后每行包含一个字符串和两个整数，分别表示学生的学号（字符串长度小于10）和出生月（ $1 \leq m \leq 12$ ）日（ $1 \leq d \leq 31$ ）。

学号、月、日之间用一个空格分隔。

输出

对每组生日相同的学生，输出一行，

其中前两个数字表示月和日，后面跟着所有在当天出生的学生的学号，数字、学号之间都用一个空格分隔。

对所有的输出，要求按日期从前到后的顺序输出。

对生日相同的学号，按输入的顺序输出。

样例输入

```

1 5
2 00508192 3 2
3 00508153 4 5
4 00508172 3 2
5 00508023 4 5
6 00509122 4 5

```

样例输出

```

1 3 2 00508192 00508172
2 4 5 00508153 00508023 00509122

```

qwen3-32b Thought for 2 minutes 16 seconds

```

1 from collections import defaultdict
2 import sys
3
4 def main():
5     n = int(sys.stdin.readline())
6     birthday_dict = defaultdict(list)
7
8     for _ in range(n):
9         data = sys.stdin.readline().strip()
10        sid, m, d = data.split()
11        month = int(m)
12        day = int(d)
13        birthday_dict[(month, day)].append(sid)
14
15    # 按日期排序
16    sorted_dates = sorted(birthday_dict.keys())
17
18    for date in sorted_dates:
19        sids = birthday_dict[date]
20        if len(sids) >= 2:
21            print(f"{date[0]} {date[1]} {' '.join(sids)}")
22
23 if __name__ == "__main__":
24     main()

```

15.74 token/s • 2334 token • 首个token用时 3.15 s • 停止原因: 检测到 EOS token

代码提交到 <http://cs101.openjudge.cn/pctbook/E02724/>, 可以AC。

3.4.2 程序API 交互

1. 确认 LM Studio API 已开启

LM Studio 提供一个 **本地 HTTP API**，默认端口是 `http://localhost:1234`。

打开 LM Studio → 点开发者按钮（左侧第一个是聊天按钮，第二个是开发者按钮），启动 **“OpenAI Compatible API Server”**。

2. 安装 Python 依赖

在终端安装官方 `openai` 库（兼容 OpenAI API 格式）。

```
1 | pip install openai requests
```

3. 编写 Python 交互脚本

首先确定哪些模型可用。在terminal中运行

```
1 | % curl http://127.0.0.1:1234/v1/models/
```

方式 A：用 `openai` 库（推荐）

LM Studio 模拟了 OpenAI API，所以可以直接用 `openai` 包。

```
1 | from openai import OpenAI
2 |
3 | # 连接到 LM Studio 的本地 API
4 | client = OpenAI(base_url="http://localhost:1234/v1", api_key="not-needed")
5 |
6 | def ask_lmstudio(prompt: str):
7 |     response = client.chat.completions.create(
8 |         model="qwen3-32b", # 模型名字可以在 LM Studio 里看到
9 |         messages=[
10 |             {"role": "system", "content": "You are a helpful AI assistant."},
11 |             {"role": "user", "content": prompt}
12 |         ],
13 |         temperature=0.7,
14 |     )
15 |     return response.choices[0].message.content
16 |
17 | if __name__ == "__main__":
18 |     question = """给出python:
19 | 在一个有180人的大班级中... (此处写完整题目) """
20 |     answer = ask_lmstudio(question)
21 |     print("模型回答: \n", answer)
```

运行：

```
1 | python chat_lmstudio.py
```

方式 B：用 `requests`（更原始）

如果不想装 `openai` 包，可以直接 POST 请求：

```
1 import requests
2 import json
3
4 API_URL = "http://localhost:1234/v1/chat/completions"
5
6 def ask_lmstudio(prompt: str):
7     headers = {"Content-Type": "application/json"}
8     data = {
9         "model": "qwen3-32b",
10        "messages": [
11            {"role": "system", "content": "You are a helpful assistant."},
12            {"role": "user", "content": prompt}
13        ],
14        "temperature": 0.7
15    }
16    response = requests.post(API_URL, headers=headers,
17                             data=json.dumps(data))
18    return response.json()["choices"][0]["message"]["content"]
19
20 if __name__ == "__main__":
21     question = "给出python：在一个有180人的大班级中..."
22     answer = ask_lmstudio(question)
23     print("模型回答：\n", answer)
```

4. 收集返回结果

- 上述代码会在终端打印结果，你也可以写入文件：

```
1 with open("answer.txt", "w", encoding="utf-8") as f:
2     f.write(answer)
```

5. 调试小技巧

- 如果 API 报错，先确认 LM Studio 设置里的 **API server** 已开启。
- 模型名称可以在 LM Studio 的 API 文档里看到（通常是你加载的模型名字，如 `qwen3-32b`）。

3.4.3 交互式版本

写一个 **交互式版本**，可以在 Terminal 连续提问，就像和模型对话一样。

这个脚本会：

- 一直循环等待用户输入
- 保留对话上下文（模型能记住之前的问答）
- 输入 `exit` 或 `quit` 就结束

```

1  from openai import OpenAI
2
3  # 连接到 LM Studio 的本地 API
4  client = OpenAI(base_url="http://localhost:1234/v1", api_key="not-needed")
5
6  def interactive_chat():
7      messages = [{"role": "system", "content": "You are a helpful AI assistant."}]
8      print("🗨️ 已连接 LM Studio, 本地模型对话开始。输入 'exit' 或 'quit' 退出。\n")
9
10     while True:
11         user_input = input("你: ").strip()
12         if user_input.lower() in ["exit", "quit"]:
13             print("👋 结束对话。")
14             break
15
16         messages.append({"role": "user", "content": user_input})
17         try:
18             response = client.chat.completions.create(
19                 model="qwen3-32b",    # 模型名称
20                 messages=messages,
21                 temperature=0.7,
22             )
23             reply = response.choices[0].message.content
24             print(f"AI: {reply}\n")
25             messages.append({"role": "assistant", "content": reply})
26         except Exception as e:
27             print("❌ 出错了:", e)
28
29 if __name__ == "__main__":
30     interactive_chat()

```

使用方式

1. 把代码保存为 `inter_chat_lmstudio.py`
2. 在 Terminal 运行:

```
1 | python inter_chat_lmstudio.py
```

3. 输入问题, 模型会回答; 输入 `exit` 或 `quit` 退出。

效果示例

```

1  🗨️ 已连接 LM Studio, 本地模型对话开始。输入 'exit' 或 'quit' 退出。
2
3  你: 给我一个冒泡排序的 Python 实现
4  AI: 好的, 这里是一个冒泡排序的示例代码.....

```


参考

1. 基于大模型和知识库构建问答教学工具, <https://pku.instructuremedia.com/embed/5741e3a7-ca03-48f6-97f0-3b5e8df42e2a>
2. 万字赏析 DeepSeek 创造之美: DeepSeek R1 是怎样炼成的? https://mp.weixin.qq.com/s?__biz=MjM5NDk5MTA0MQ==&mid=2652324720&idx=1&sn=5ce38a508dc0ae33a870964aea367e2e&scene=21#wechat_redirect
3. In 2024, the LLM field saw increasing specialization. Beyond pre-training and fine-tuning, we witnessed the rise of specialized applications, from RAGs to code assistants. I expect this trend to accelerate in 2025, with an even greater emphasis on domain- and application-specific optimizations (i.e., "specializations"). <https://magazine.sebastianraschka.com/p/understanding-reasoning-llms>
4. Build a Large Language Model (From Scratch) (Sebastian Raschka) 这本书的程序, 可以在 <https://colab.research.google.com/> 用免费的GPU运行。 <https://github.com/rasbt/LLMs-from-scratch>
5. DeepSeek R1 technical report, <https://arxiv.org/abs/2501.12948>.